UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

SCUOLA POLITECNICA E DELLE SCIENZE DI BASE

DIPARTIMENTO DI INGEGNERIA ELETTRICA E TECNOLOGIE DELL'INFORMAZIONE

Corso di Laurea Magistrale in Ingegneria Informatica

ELABORATO DI NETWORKS AND CLOUD INFRASTRUCTURES

Prof. Giorgio Ventre

a.a. 2023-24

Studenti:

Migliaccio Luca – M63/1573

Migliuolo Mario – M63/1617

Indice

Capitolo 1: Analisi del Problema 3
1.1 Introduzione al Problema 3
1.2 DoS – Denial of Service 3
1.3 Obiettivi del Progetto 3
Capitolo 2: Preparazione dell'ambiente 4
2.1 Configurazione della VM 4
2.2 Installazione di Mininet 4
Capitolo 3: Progettazione della Topologia di Rete 4
3.1 Definizione della Topologia e Architettura di Rete 4
3.2 Script Topologia 6
3.3 Script Controller 8
Capitolo 4: Generazione ed Analisi del Traffico 11
4.1 Configurazione Host lato Server 11
4.2 Generazione/Invio del Traffico lato Client 12
4.3 Analisi del Traffico (senza mitigazione) 13
Capitolo 5: Tecniche per la Mitigazione degli Attacchi 15
5.1 Topologia 1: soluzione 15
5.1.1 Modifiche controller 16
5.1.2 Mitigazione dell'attacco 20
5.2 Topologia 2: soluzioni 23
5.2.1 Script topologia 24
5.2.2 Soluzione 1 – Estensione alla nuova topologia 25
5.2.3 Soluzione 1 – Introduzione alla generazione di traffico variabile 28
5.2.4 Soluzione 2 – Approccio Dinamico 31
5.2.5 Soluzione 2 – Esecuzione con traffico costante 32
5.2.6 Soluzione 2 – Esecuzione con traffico variabile 36
5.2.7 Soluzione 2 – Rimozione Link S1-S3 41
5.2.8 Soluzione Alternativa – Blocco dell'indirizzo IP 44
5.2.9 Soluzione Alternativa – Esecuzione con traffico variabile 47
Conclusioni & Considerazioni 50
Approcci alternativi 51
Riferimenti 51

Capitolo 1: Analisi del Problema

In questo capitolo si affronterà il problema critico della sicurezza di rete, evidenziando la necessità di un sistema robusto di monitoraggio e mitigazione degli attacchi **Denial of Service – DoS**. Successivamente, si definiranno gli obiettivi principali del progetto, delineando le sfide e le aspettative da affrontare durante l'implementazione del **sistema SDN Firewall**.

1.1 Introduzione al Problema

La sicurezza di rete è una preoccupazione primaria in un ambiente informatico sempre più interconnesso e vulnerabile agli attacchi informatici. Tra le minacce più comuni, gli attacchi DoS si distinguono per la loro capacità di sovraccaricare i dispositivi di rete o i servizi online, compromettendo la disponibilità e il funzionamento regolare dei servizi. Tali attacchi possono essere perpetrati da singoli attaccanti o da reti di dispositivi compromessi, comportando perdite finanziarie, danni all'immagine aziendale e interruzioni dei servizi critici.

La complessità e la scala degli attacchi DoS richiedono un sistema di difesa avanzato in grado di rilevare tempestivamente le anomalie nel traffico di rete e di intraprendere azioni correttive per mitigare l'impatto negativo sugli utenti legittimi. In questo contesto, l'adozione di tecnologie di rete definibili dal software (SDN) offre un approccio innovativo alla sicurezza di rete, consentendo una gestione centralizzata e dinamica delle risorse di rete e delle politiche di sicurezza.

1.2 DoS – Denial of Service

Un **attacco Denial-of-Service (DoS)** nelle reti *OpenFlow SDN* comporta un *sovraccarico* delle risorse di calcolo o di rete tale da impedire a uno switch di inoltrare i pacchetti come previsto. Un attacco riuscito comporta l'invio di un gran numero di pacchetti allo switch, eventualmente istanziando nuovi flussi.

1.3 Obiettivi del Progetto

Il presente progetto mira a sviluppare un sistema SDN Firewall che abbia come **obiettivo primario** la protezione della rete da attacchi DoS. Gli obiettivi principali del progetto includono:

- *Monitoraggio del Traffico*: Implementare un sistema di monitoraggio del traffico di rete in grado di rilevare pattern anomali e segnalare potenziali attacchi DoS.
- *Mitigazione degli Attacchi*: Progettare e implementare un meccanismo di mitigazione automatica degli attacchi DoS, che possa identificare e bloccare il traffico dannoso per preservare la disponibilità dei servizi di rete.
- *Valutazione delle Prestazioni di Rete*: Condurre un'analisi delle prestazioni di rete al fine di valutare l'efficacia delle soluzioni implementate e identificare eventuali aree di miglioramento.

Attraverso la realizzazione di tali obiettivi, ci si propone di fornire una soluzione efficace e scalabile per proteggere le reti da attacchi DoS, garantendo al contempo la continuità operativa e la sicurezza dei servizi di rete.

Capitolo 2: Preparazione dell'ambiente

2.1 Configurazione della VM

Per la parte di **setup** è stato necessario creare una **VM Ubuntu**. In particolare, è stata utilizzata la versione Ubuntu 22, tramite Virtualbox. Successivamente, sono stati installati i pacchetti necessari alla realizzazione del progetto eseguendo i seguenti comandi:

- *sudo apt install vim*
- *sudo apt install git*
- *sudo apt install python3-pip*
- *sudo pip install ryu*
- *sudo apt install d-itg*
- *sudo apt install nload*

2.2 Installazione di Mininet

Mininet è un emulatore leggero basato su virtualizzazione e contenitori che permette di riprodurre reti virtuali con topologie arbitrarie:

- requisiti hardware modesti, avvio veloce, centinaia di nodi
- strumento a riga di comando, CLI, semplice API Python

Mininet consente lo sviluppo di SDN su qualsiasi laptop o PC e i progetti SDN possono spostarsi senza problemi tra Mininet (consentendo uno sviluppo economico e semplificato) e l'hardware reale che funziona a velocità di linea nelle distribuzioni live.

Gli script Python possono essere utilizzati per orchestrare un esperimento

- Definizione della topologia di rete
- Eventi da attivare nei nodi della rete (ad esempio l'esecuzione di un programma)

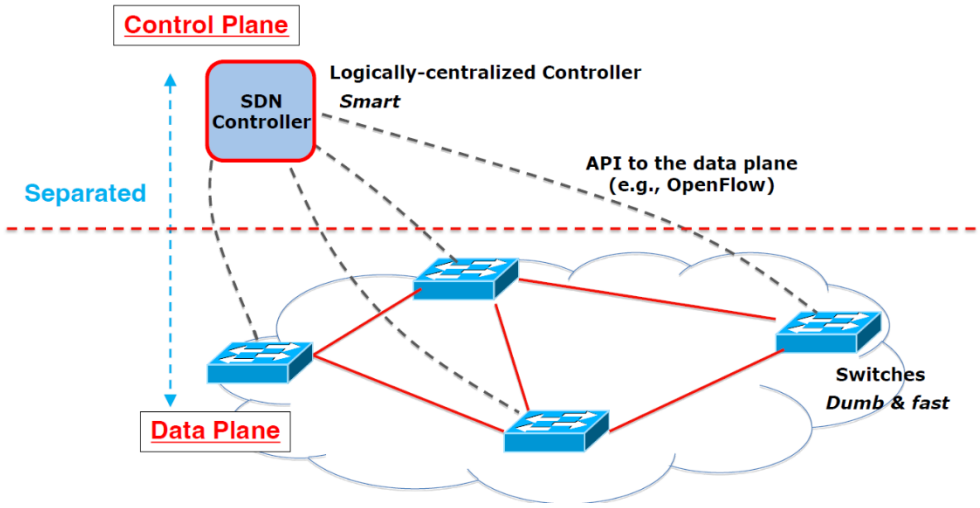
Installazione di Mininet VM: il modo più semplice per installare Mininet

1. Scaricare e installare sul proprio PC uno degli hypervisor disponibili (ad esempio, VirtualBox o KVM).
2. Scaricare l'immagine della macchina virtuale Mininet preinstallata.
3. Importare l'immagine della macchina virtuale nell'hypervisor selezionato

Capitolo 3: Progettazione della Topologia di Rete

3.1 Definizione della Topologia e Architettura di Rete

- **Data Plane**: È composto da **switch** e **host**. La funzione principale del data plane è inoltrare i pacchetti di rete basati sulle regole di inoltro configurate dal controller. Gli switch in questo piano gestiscono la trasmissione dei pacchetti tra host e switch secondo le tabelle di forwarding.
- **Control Plane**: È rappresentato dal **controller** (c0). Il controller è responsabile della gestione delle tabelle di inoltro sugli switch e dell'implementazione della logica di rete. In questa topologia, c0 è un controller remoto che comunica con gli switch per definire come i pacchetti devono essere inoltrati.



La topologia di rete in questione mostra una configurazione di dispositivi e collegamenti in cui l'host H1 è un attaccante che tenta di effettuare un attacco Denial of Service (DoS). La topologia comprende i seguenti *componenti*:

1. **Host (H)**:
    - **H1**: è l'attaccante ed ha l'obiettivo di effettuare un attacco DoS.
    - **H2**: è un host regolare che si collega allo switch S2.
    - **H3**: è un host regolare che si collega allo switch S4.
2. **Switch (S)**:
    - **S1**: collegato a H1.
    - **S2**: collegato a H2.
    - **S3**: collegato a S1, S2 e S4 (questo switch funge da nodo centrale di connessione).
    - **S4**: collegato a S3 e H3.
3. **Controller (C)**:
    - Il controller C gestisce l'intera rete, configurando e monitorando gli switch per instradare correttamente il traffico.

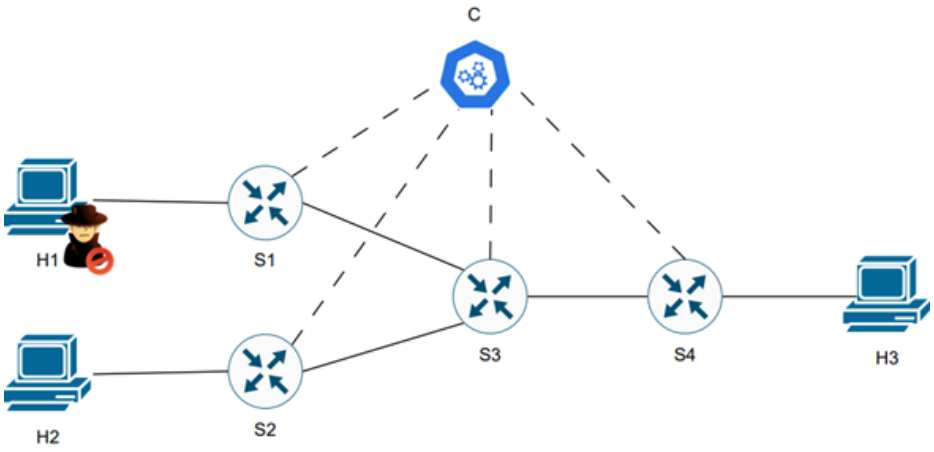
Nella descrizione della topologia, sono presenti anche i *collegamenti* tra i componenti. In particolare, abbiamo:

- **link H1 - S1**
- **link H2 - S2**
- **link S1 - S3**
- **link S2 - S3**
- **link S3 - S4**
- **link S4 - H3**

N.B. Tutti gli switch (S1, S2, S3, S4) sono collegati al controller (C) per il controllo e la gestione.

L'host H1, essendo l'attaccante, tenterà di generare una grande quantità di traffico verso H3 causando congestione sui collegamenti, esaurendo le risorse di rete o i buffer di memoria dei dispositivi, causando un'anomalia nel traffico dati. Viceversa, H2 genera traffico normale verso H3.

N.B. H1 genera traffico UDP, mentre H2 genera traffico TCP, entrambi hanno bitrate costante.



3.2 Script Topologia

Riportiamo di seguito lo script python della topologia descritta nei paragrafi precedenti:

topology_v2.py > ...

```python
#!/usr/bin/python

# importazione librerie
import random
import threading
import time
from mininet.net import Mininet
from mininet.node import RemoteController, OVSKernelSwitch, Host
from mininet.link import TCLink
from mininet.cli import CLI
from mininet.log import setLogLevel, info

class Environment(object):
    @staticmethod
    def myTopology():
        # crea un'istanza di Mininet per la simulazione della rete
        net = Mininet(controller=RemoteController, link=TCLink, switch=OVSKernelSwitch)

        # aggiuta di un controller alla rete
        info('*** Adding controller\n')
        c0 = net.addController(name='c0', controller=RemoteController)
        c0.start()

        # aggiuta degli host alla rete
        info('*** Adding hosts\n')
        H1 = net.addHost('h1', cls=Host, mac='00:00:00:00:00:01', ip='10.0.0.1')
        H2 = net.addHost('h2', cls=Host, mac='00:00:00:00:00:02', ip='10.0.0.2')
        H3 = net.addHost('h3', cls=Host, mac='00:00:00:00:00:03', ip='10.0.0.3')

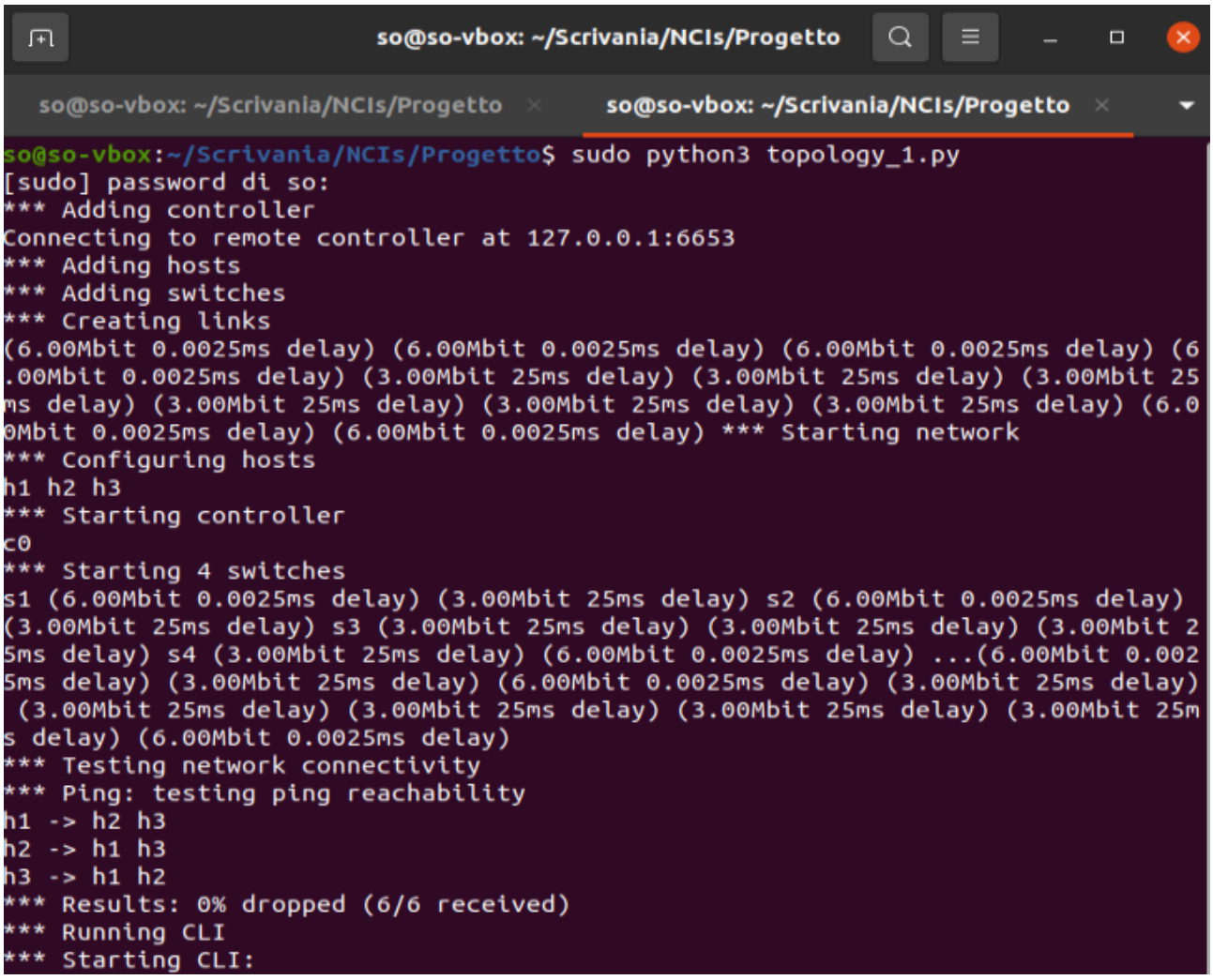
        # aggiuta degli switch alla rete
        info('*** Adding switches\n')
        S1 = net.addSwitch('s1', cls=OVSKernelSwitch, protocols='OpenFlow13')
        S2 = net.addSwitch('s2', cls=OVSKernelSwitch, protocols='OpenFlow13')
        S3 = net.addSwitch('s3', cls=OVSKernelSwitch, protocols='OpenFlow13')
        S4 = net.addSwitch('s4', cls=OVSKernelSwitch, protocols='OpenFlow13')

```

```
37          # aggiuta dei link tra gli host e gli switch alla rete
38          info('*** Creating links\n')
39          net.addLink(H1, S1, bw=6, delay='0.0025ms')
40          net.addLink(H2, S2, bw=6, delay='0.0025ms')
41          net.addLink(S1, S3, bw=3, delay='25ms')
42          net.addLink(S2, S3, bw=3, delay='25ms')
43          net.addLink(S3, S4, bw=3, delay='25ms')
44          net.addLink(S4, H3, bw=6, delay='0.0025ms')
45
46          info('*** Starting network\n')
47          net.build()
48          net.start()
49
50          info('*** Testing network connectivity\n')
51          net.pingAll()
52
53          info('*** Running CLI\n')
54          CLI(net)
55
56          info('*** Stopping network\n')
57          net.stop()
58
59  # avvio dell'ambiente di rete e la CLI di Mininet
60  if __name__ == '__main__':
61      setLogLevel('info')
62      Environment.myTopology()
63
```

Osservazioni:

- *net.addController* → specifichiamo il nome del controller (univoco) e la classe (controller). Per quanto riguarda la classe, specificando **RemoteController** possiamo eseguire il controller come un processo separato.
- *net.addHost* → specifichiamo il nome dell'host, la classe (cls), l'indirizzo MAC (univoco) e l'indirizzo IP (univoco). Come classe abbiamo utilizzato quella di default, ovvero **Host**.
- *net.addSwitch* → specifichiamo il nome dello switch, la classe (cls) ed il protocollo di controllo del flusso che lo switch supporta (in questo caso il protocollo usato è **OpenFlow13**).
- *net.addlink* → specifichiamo i nodi tra cui stabilire il collegamento, la larghezza di banda (bw) e il delay, ovvero il ritardo (in ms) su quel collegamento.
- *info(…)* → serve per stampare messaggi informativi.
- *net.build( )* → costruisce la topologia di rete.
- *net.start( )* → avvia la rete.
- *net.pingAll( )* → comando utilizzato per testare la connettività della rete appena realizzata, andando ad eseguire un comando ping tra tutti i nodi della rete (vedi immagine seguente).
- *CLI(net)* → avvia la CLI (interfaccia a riga di comando – Command Line Interface) per permettere l'interazione con la rete.
- *net.stop( )* → ferma la rete.
- *setLogLevel('info')* → imposta il livello di log su 'info' per visualizzare i messaggi informativi.
- *Environment.myTopology( )* → crea ed avvia la topologia definita nella classe Environment.



N.B. per avviare la topologia, il comando è *'sudo python3 tipology_1.py'* (come indicato nell'immagine precedente).

3.3 Script Controller

Riportiamo di seguito lo script python del controller fornito durante il corso, ovvero senza alcuna mitigazione:

```python
from ryu.base import app_manager
from ryu.controller import ofp_event
from ryu.controller.handler import CONFIG_DISPATCHER, MAIN_DISPATCHER
from ryu.controller.handler import set_ev_cls
from ryu.ofproto import ofproto_v1_3
from ryu.lib.packet import packet
from ryu.lib.packet import ethernet
from ryu.lib.packet import ether_types

class SimpleSwitch13(app_manager.RyuApp):

    # Specifica la versione OpenFlow supportata dall'applicazione
    OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]

    def __init__(self, *args, **kwargs):
        # Inizializza l'applicazione
        super(SimpleSwitch13, self).__init__(*args, **kwargs)

        # Dizionario per tenere traccia dei MAC address associati alle porte
        self.mac_to_port = {}

    # Gestisce gli eventi di features dello switch
    @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
    def switch_features_handler(self, ev):
        # Ottiene le informazioni necessarie dal messaggio dell'evento
        datapath = ev.msg.datapath
        ofproto = datapath.ofproto
        parser = datapath.ofproto_parser

        # Installa una regola per gestire i pacchetti che non corrispondono a nessuna regola
        match = parser.OFPMatch()
        actions = [parser.OFPActionOutput(ofproto.OFPP_CONTROLLER, ofproto.OFPCML_NO_BUFFER)]
        self.add_flow(datapath, 0, match, actions)

    # Aggiunge una regola di flusso
    def add_flow(self, datapath, priority, match, actions, buffer_id=None):
        ofproto = datapath.ofproto
        parser = datapath.ofproto_parser

```

```
        # Specifica le azioni da eseguire per il flusso
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS, actions)]

        # Crea un messaggio di modifica del flusso e lo invia allo switch
        if buffer_id:
            mod = parser.OFPFlowMod(datapath=datapath, buffer_id=buffer_id, priority=priority, match=match, instructions=inst)
        else:
            mod = parser.OFPFlowMod(datapath=datapath, priority=priority, match=match, instructions=inst)
        datapath.send_msg(mod)

    # Gestisce gli eventi di pacchetti in arrivo
    @set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
    def _packet_in_handler(self, ev):
        # Ottiene le informazioni necessarie dal messaggio dell'evento
        msg = ev.msg
        datapath = msg.datapath
        ofproto = datapath.ofproto
        parser = datapath.ofproto_parser
        in_port = msg.match['in_port']

        # Ottiene l'oggetto del pacchetto Ethernet
        pkt = packet.Packet(msg.data)
        eth = pkt.get_protocols(ethernet.ethernet)[0]

        # Ignora i pacchetti LLDP
        if eth.ethertype == ether_types.ETH_TYPE_LLDP:
            return

        # Ottiene l'indirizzo MAC di destinazione e sorgente
        dst = eth.dst
        src = eth.src
        dpid = datapath.id
        self.mac_to_port.setdefault(dpid, {})  # Inizializza il dizionario se non esiste già

        # Logga le informazioni sul pacchetto in arrivo
        self.logger.info("packet in %s %s %s %s", dpid, src, dst, in_port)
```

```
        # Memorizza l'associazione MAC-porta
        self.mac_to_port[dpid][src] = in_port

        # Trova la porta di destinazione o invia in flooding se sconosciuta
        if dst in self.mac_to_port[dpid]:
            out_port = self.mac_to_port[dpid][dst]
        else:
            out_port = ofproto.OFPP_FLOOD

        # Specifica le azioni da eseguire sul pacchetto
        actions = [parser.OFPActionOutput(out_port)]

        # Installa una regola di flusso se non è un pacchetto di flooding
        if out_port != ofproto.OFPP_FLOOD:
            match = parser.OFPMatch(in_port=in_port, eth_dst=dst, eth_src=src)
            if msg.buffer_id != ofproto.OFP_NO_BUFFER:
                self.add_flow(datapath, 1, match, actions, msg.buffer_id)
                return
            else:
                self.add_flow(datapath, 1, match, actions)

        # Invia il pacchetto alla porta di destinazione
        data = None
        if msg.buffer_id == ofproto.OFP_NO_BUFFER:
            data = msg.data

        out = parser.OFPPacketOut(datapath=datapath, buffer_id=msg.buffer_id, in_port=in_port, actions=actions, data=data)
        datapath.send_msg(out)
```

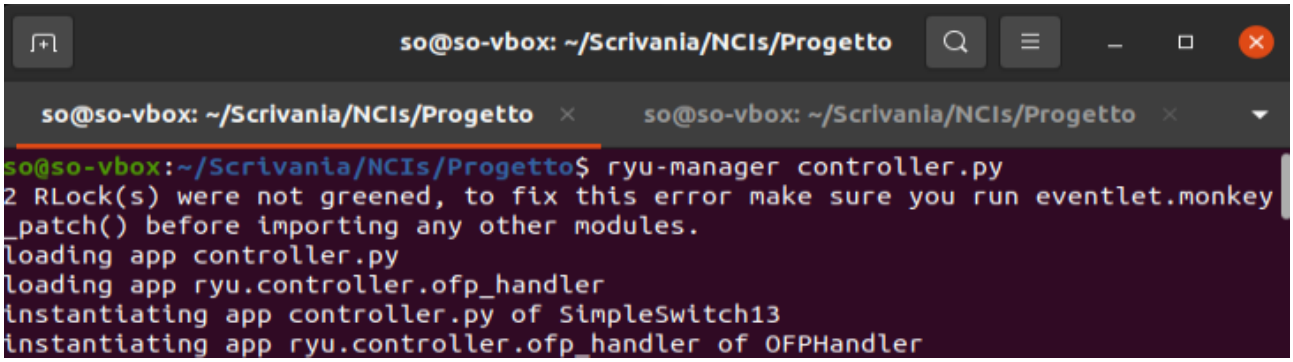
Osservazioni:

- *def switch_features_handler* → chiamata quando lo switch invia un messaggio *Features Reply*. Viene utilizzata per **impostare** una regola di flusso di default che invia tutti i pacchetti che non corrispondono a nessuna regola al controller.
    - **datapath**: rappresenta lo switch
    - **ofproto**: contiene le costanti del protocollo OpenFlow
    - **parser**: fornisce i metodi per creare messaggi OpenFlow
    - **match**: crea una corrispondenza vuota che corrisponde a tutti i pacchetti
    - **actions**: specifica un'azione per inviare i pacchetti al controller

- *def add_flow* → aggiunge una regola di flusso (flow entry) allo switch con criteri di corrispondenza (come, ad esempio, una certa priorità) e azioni specificate.
    - **priority**: priorità della regola
    - **buffer_id**: se specificato, contiene l'ID del buffer del pacchetto da cui è stata generata la richiesta
    - **inst**: crea un'istruzione per applicare le azioni specificate
    - **mod**: crea un messaggio di modifica del flusso (FlowMod) e lo invia allo switch

- *def _packet_in_handler* → è un gestore di eventi che viene chiamato quando uno switch OpenFlow invia un messaggio *PacketIn* al controller. Questo gestore permette al controller di **gestire** pacchetti che non corrispondono a nessuna regola di flusso esistente sullo switch. In particolare, gestisce i pacchetti in arrivo dagli switch, aggiorna la tabella MAC-porta, determina la porta di uscita appropriata e installa regole di flusso per gestire i pacchetti futuri con gli stessi criteri di corrispondenza. Se l'indirizzo MAC di destinazione è sconosciuto, il pacchetto viene inviato in flooding.
    - **in_port**: è la porta di ingresso sulla quale arriva il pacchetto
    - **pkt**: decodifica i dati del pacchetto ricevuto
    - **eth**: estrae il protocollo Ethernet dal pacchetto
    - **dst** e **src**: sono gli indirizzi MAC di destinazione e sorgente
    - **dpid**: ID del datapath dello switch

Per avviare il controller utilizziamo il seguente comando: '*ryu-manager controller.py*':



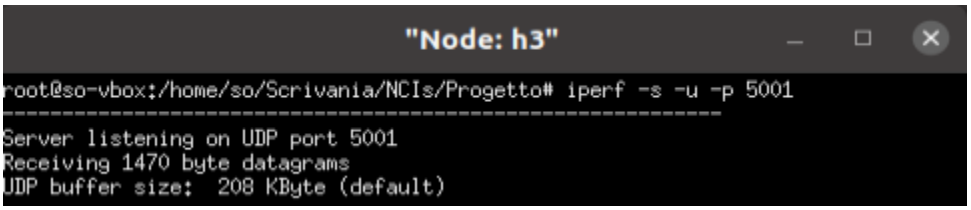
N.B. è necessario avviare prima il controller e poi, successivamente, la topologia. Questo perché nella topologia si va a creare un controller di classe **RemoteController**.

Capitolo 4: Generazione ed Analisi del Traffico

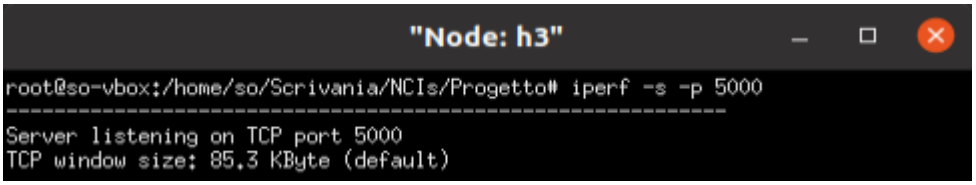
4.1 Configurazione Host lato Server

Per stabilire una comunicazione tra gli host all'interno della topologia, la prima cosa da fare è configurare l'host h3. Tale device viene messo in *modalità server (-s)*, ricevendo traffico TCP e UDP, tramite il comando *iperf*. Andiamo ad aprire con il comando *xterm h3 h3* due interfacce differenti, una per ogni host con il quale si sta comunicando.

La prima comunicazione viene aperta sulla **porta 5001** (*-p 5001*), utilizzando il comando riportato in figura, per permettere ad h3 di ricevere il traffico dall'host h1 (di tipo **UDP** → *-u*).



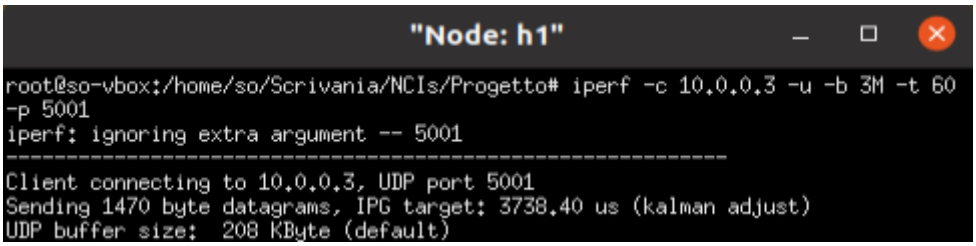
La seconda comunicazione, invece, viene aperta sulla **porta 5000** (*-p 5000*), il cui comando è riportato nella figura seguente. Come richiesto dalla traccia, permette ad h3 di ricevere il traffico dall'host h2 (di tipo **TCP**).



N.B. il protocollo predefinito per la seconda comunicazione è TCP, dato che -u non è specificato.

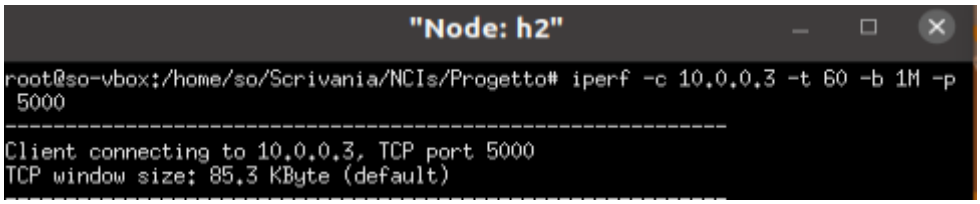
4.2 Generazione/Invio del Traffico lato Client

Per effettuare una comunicazione tra h1-h3 e h2-h3, i comandi utilizzati sono riportati di seguito:



Osservazioni:

- *-c 10.0.0.3*: si esegue il test come client e si effettua la connessione con l'indirizzo IP 10.0.0.3
- *-u*: uso UDP
- *-b 3M*: impostata la larghezza di banda a 3 Megabit/s
- *-t 60*: si esegue il test per 60 secondi
- *-p 5001*: si usa la porta 5001 per la connessione al server



Osservazioni:

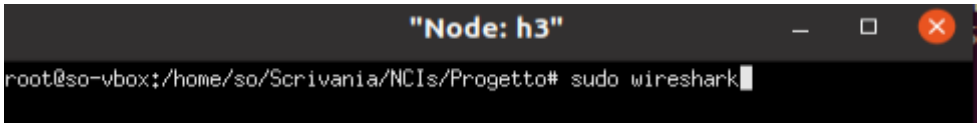
- *-c 10.0.0.3*: si esegue il test come client e si effettua la connessione con l'indirizzo IP 10.0.0.3
- *-b 1M*: impostata la larghezza di banda a 1 Megabit/s

- *-t 60*: si esegue il test per 60 secondi
- *-p 5000*: si usa la porta 5000 per la connessione al server

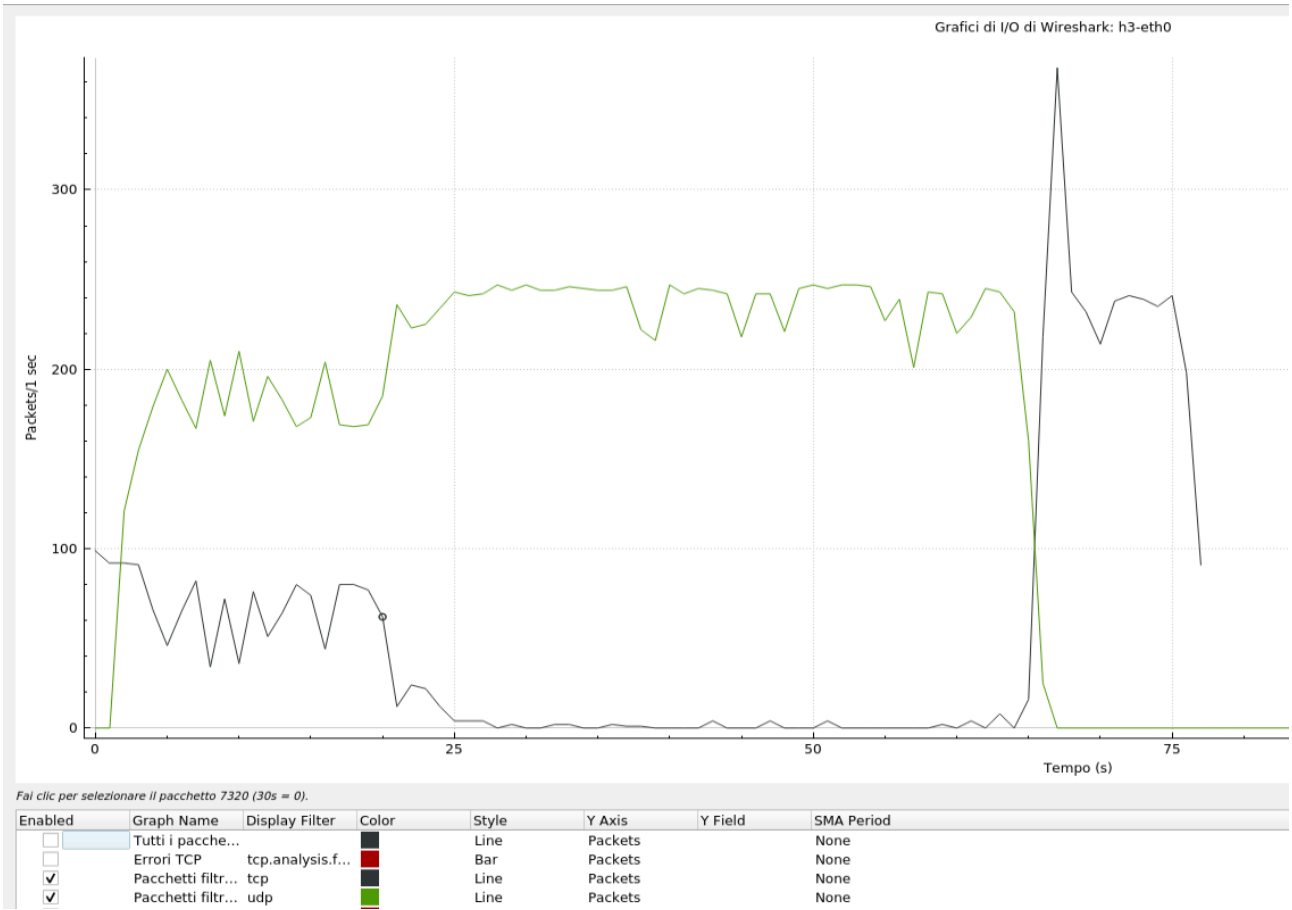
4.3 Analisi del Traffico (senza mitigazione)

Non avendo inserito all'interno del controller nessuna strategia di mitigazione, i risultati che sono stati visualizzati (utilizzando **wireshark**) sono riportati di seguito. In particolare, sono stati utilizzati differenti filtri, in maniera tale da comprendere le diverse tipologie di traffico.

Innanzitutto, abbiamo aperto un ulteriore terminale utilizzando *xterm h3* e, successivamente, abbiamo eseguito il comando in figura:

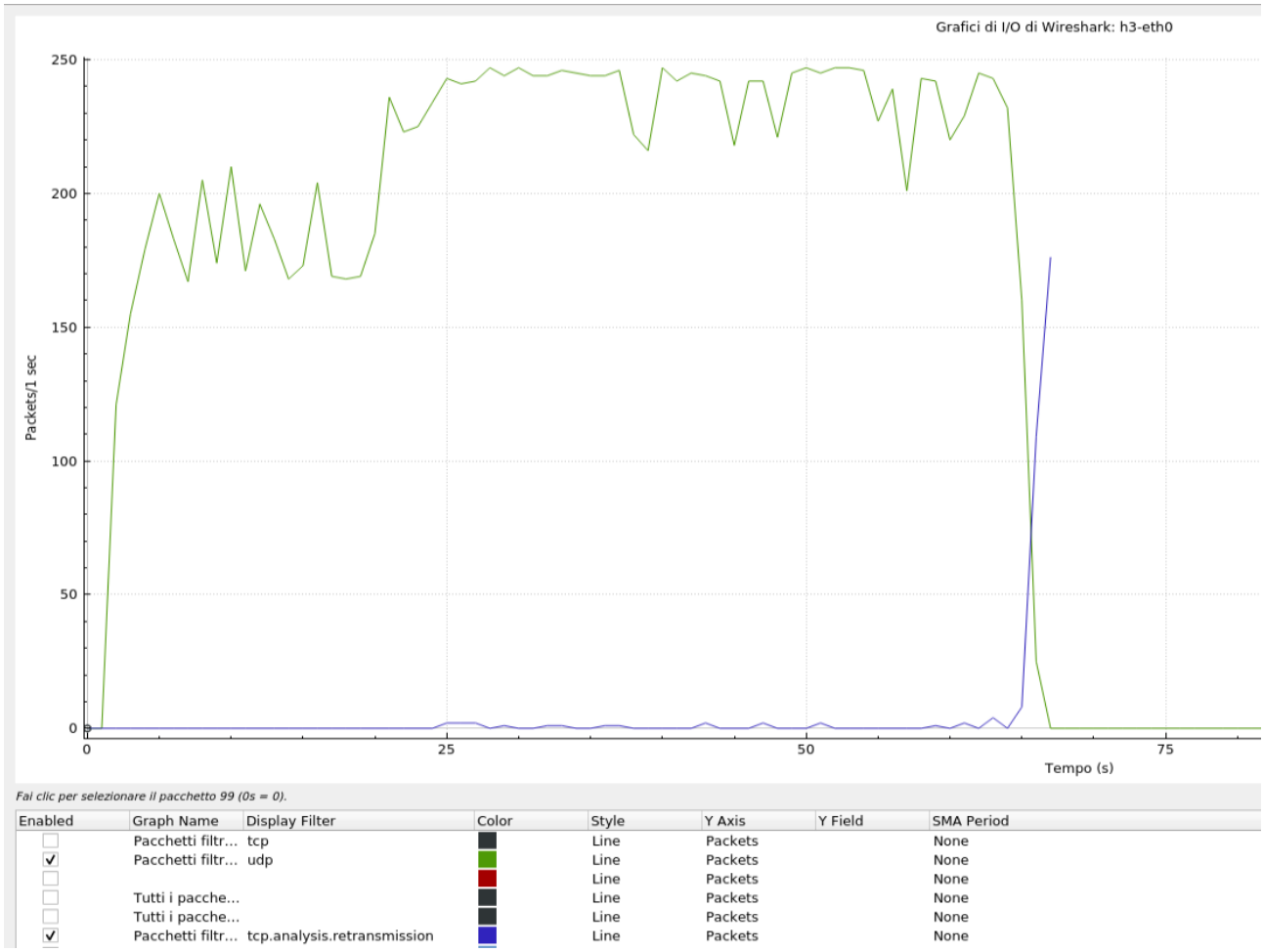


- **Filtro TCP/UDP:**



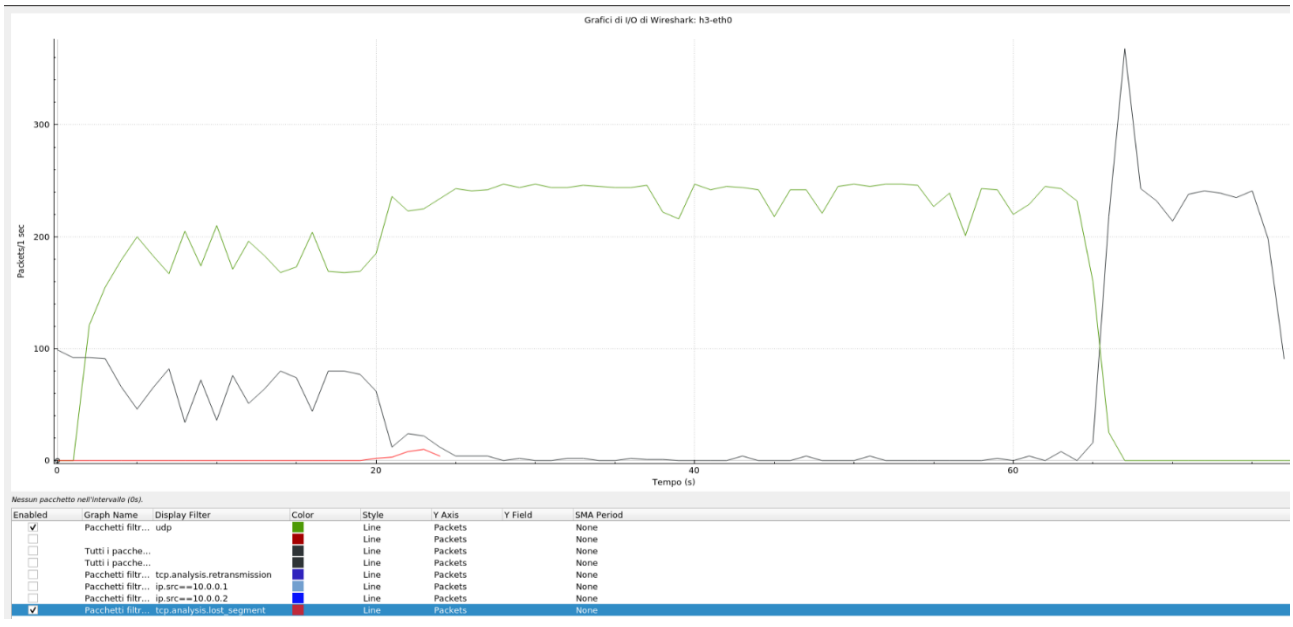
Dai seguenti grafici si può osservare che, nel momento in cui l'attacco è in corso (traffico UDP elevato), l'host h2 (traffico TCP) viene sovrastato, causando l'interruzione della comunicazione tra h2 e h3. Una volta che l'attacco è terminato (dopo ~ 55 secondi), il traffico TCP cresce, ristabilendo una corretta comunicazione tra gli host h2 e h3.

- **Filtro tcp.analysis.retransmission:**



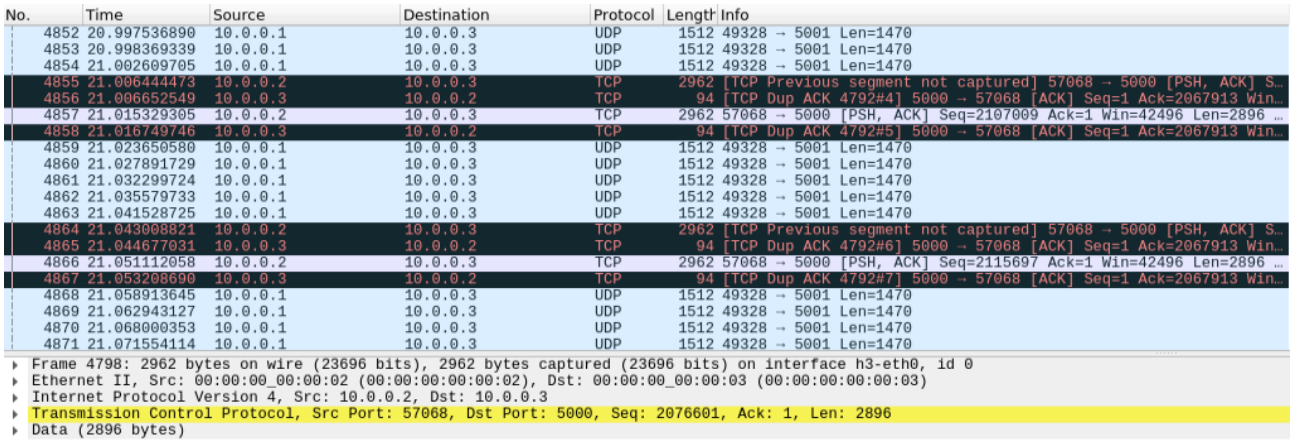
Dal seguente grafico, possiamo notare come la ritrasmissione dei pacchetti riprenda esattamente nel momento in cui si ha un calo drastico del traffico UDP (come osservato precedentemente).

- **Filtro tcp.analysis.lost_segment**



Come mostrato nel grafico precedente, la perdita dei segmenti TCP si ha nel momento in cui si ha il sopravvento dei pacchetti UDP rispetto a quelli TCP (si inizia ad osservare un aumento significativo intorno ai 20 secondi).

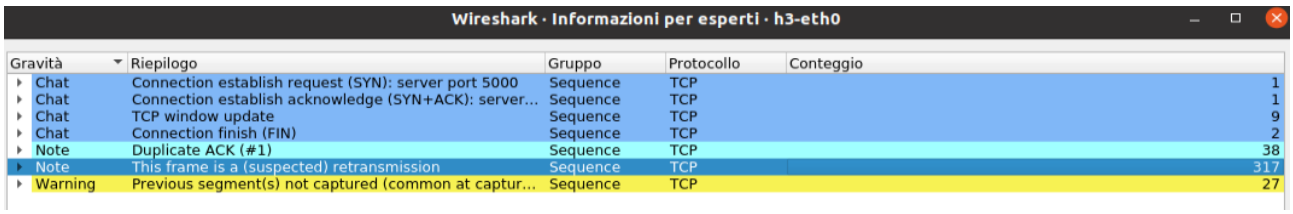
Di seguito è riportata una porzione dell'attività di sniffing effettuata, dove è possibile notare l'effettiva ritrasmissione dei pacchetti TCP (indicati in nero):



In particolare, tra le diverse informazioni, abbiamo:

- *Ritrasmissioni TCP*
- *Conferme duplicate*: **TCP Dup ACK** si ha quando alcuni pacchetti sono stati persi e devono essere ritrasmessi
- *Segmenti mancanti*: **TCP Previous segment not captured** si ha quando alcuni segmenti precedenti non sono stati catturati

Nella seguente immagine sono riportate una serie di *informazioni avanzate*:

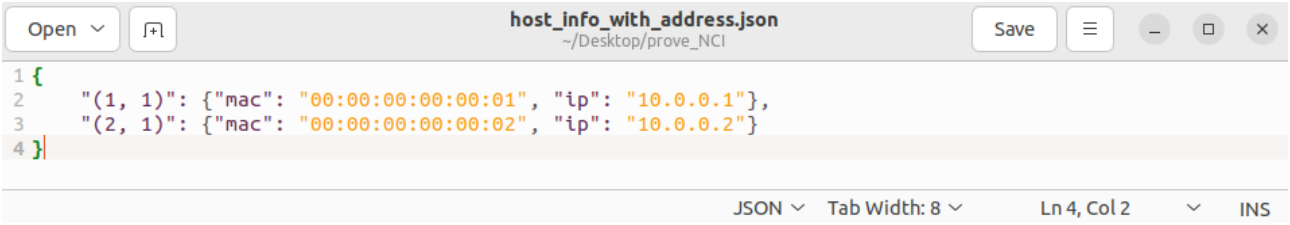


Capitolo 5: Tecniche per la Mitigazione degli Attacchi

5.1 Topologia 1: soluzione

L'approccio utilizzato, al fine di bloccare correttamente solo la porta dell'host attaccante, senza bloccare (in cascata) le porte intermedie, si è basato su di un file *json* (JavaScript Object Notation) in cui sono state riportate solo le informazioni degli host (che trasmettono), ovvero: *switch, porta di collegamento tra porta e switch, indirizzo mac* ed *indirizzo ip*. È stato ritenuto opportuno l'utilizzo di questo file, in quanto, il file che descrive la topologia di rete è in nostro possesso e abbiamo completa visione di quest'ultimo (il file in formato

json possiede solo alcune informazioni ricavabili tranquillamente dalla topologia). Ecco il contenuto del file utilizzato per la prima soluzione:



5.1.1 Modifiche controller

Di seguito riportiamo il codice (solo le parti differenti dal controller originale – senza mitigazione):

```python
import threading
import time
import json

from ryu.base import app_manager
from ryu.controller import ofp_event
from ryu.controller.handler import CONFIG_DISPATCHER, MAIN_DISPATCHER, set_ev_cls
from ryu.ofproto import ofproto_v1_3
from ryu.lib.packet import packet, ethernet, ether_types

class SimpleSwitch13(app_manager.RyuApp):
    OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]

    def __init__(self, *args, **kwargs):
        super(SimpleSwitch13, self).__init__(*args, **kwargs)
        self.mac_to_port = {}
        self.datapaths = {}
        self.stats_interval = 5 # Intervallo per raccogliere le statistiche dei porti (in secondi)
        self.threshold = 2 * (10**6) # Soglia in bit per secondo (2 Mbps)
        self.alarm = {}
        self.blocked_ports = set()
        self.entry_ports = {} # Dizionario per tenere traccia delle porte di ingresso per ogni flusso
        self.block_duration = 10 # Durata del blocco in secondi (prima prova: 5)
        self.prev_stats = {}
        self.host_info = {} # Dizionario per tenere traccia degli host e delle loro porte e switch

        self.load_host_info_json("host_info_with_address.json")
        #self.load_host_info_json("host_info_net2.json")

    def load_host_info_json(self, filepath):
        with open(filepath, 'r') as json_file:
            data = json.load(json_file)
            self.host_info = {eval(key): value for key, value in data.items()} # per convertire le chiavi da stringhe a tuple

            print("\n--- DEBUG: STAMPA CONTENUTO FILE JSON ---")
            print(self.host_info)
            print("----------------\n")

```

Osservazioni:

- *Dizionari e variabili*:
    - **self.mac_to_port**: dizionario per mappare gli indirizzi MAC (per evitare il flooding)
    - **self.datapaths**: dizionario per memorizzare i datapaths
    - **self.stats_interval**: intervallo (in secondi) per la raccolta delle statistiche dei porti
    - **self.threshold**: soglia misurata in bit/sec
    - **self.alarm**: dizionario per tenere traccia di un eventuale superamento della soglia
    - **self.blocked_ports:** dizionario per tenere traccia delle porte bloccate (**set** per evitare la presenza di duplicati)
    - **self.entry_ports**: dizionario per tenere traccia delle porte di ingresso per ogni flusso
    - **self.block_duration**: durata (in secondi) del blocco della porta
    - **self.prev_stats:** dizionario per il calcolo delle statistiche, in quanto la raccolta di quest'ultime è cumulativa. Al fine di ottenere i valori corretti è necessario calcolare il *valore corrente = nuovo valore – prev_stats*

- **self.host_info**: dizionario per tenere traccia delle informazioni contenute nel file json (host, porte, switch)

- *def load_host_info_json* → serve per la lettura delle informazioni dal file json e per il loro inserimento nel dizionario host_info.

```
70      def start_port_stats_collection(self, datapath):
71          # Implementa la logica per iniziare la raccolta delle statistiche dei porti
72          threading.Thread(target=self.collect_port_stats, args=(datapath,)).start()
73
74      def collect_port_stats(self, datapath):
75          while True:
76              ofproto = datapath.ofproto
77              parser = datapath.ofproto_parser
78
79              req = parser.OFPPortStatsRequest(datapath, 0, ofproto.OFPP_ANY)
80              datapath.send_msg(req)
81              time.sleep(self.stats_interval)
```

Osservazioni:

- *def start_port_stats_collection* → implementa la logica per iniziare la raccolta delle statistiche, lanciando un Thread.
- *def collect_port_stats* → eseguita in un thread separato, si occupa di raccogliere periodicamente le statistiche delle porte di uno switch.
    - **OFPPortStatsRequest**: inviato periodicamente per richiedere le statistiche delle porte al datapath specificato
    - **time.sleep(self.stats_interval)**: serve per inviare la richiesta ogni n (stats_interval) secondi

```
83      @set_ev_cls(ofp_event.EventOFPPortStatsReply, MAIN_DISPATCHER)
84      def port_stats_reply_handler(self, ev):
85          body = ev.msg.body
86          dpid = ev.msg.datapath.id
87
88          if dpid not in self.datapaths:
89              return
90
91          datapath = self.datapaths[dpid]
92
93          for stat in body:
94              if stat.port_no == ofproto_v1_3.OFPP_LOCAL:
95                  continue
96
97              port_no = stat.port_no
98              current_rx_bytes = stat.rx_bytes
99              current_tx_bytes = stat.tx_bytes
100
101             if (dpid, port_no) in self.prev_stats:
102                 prev_rx_bytes, prev_tx_bytes = self.prev_stats[(dpid, port_no)]
103                 rx_bytes_diff = current_rx_bytes - prev_rx_bytes
104                 tx_bytes_diff = current_tx_bytes - prev_tx_bytes
105
106                 rx_throughput = (rx_bytes_diff * 8) / self.stats_interval
107                 tx_throughput = (tx_bytes_diff * 8) / self.stats_interval
108
109                 self.prev_stats[(dpid, port_no)] = (current_rx_bytes, current_tx_bytes)
110
111                 if rx_throughput > self.threshold:
112                     self.alarm[dpid, port_no] = True
113                     self.logger.info("-----------SUPERO LA SOGLIA----------")
114
115                     if (dpid, port_no) not in self.blocked_ports:
116                         if (dpid, port_no) not in self.entry_ports:
117                             self.logger.info("****** CONTROLLO SE CHIAMARE LA FUNZIONE PER BLOCCARE IL TRAFFICO ******")
118                             self.check_and_block_port(datapath, port_no)
119                             self.entry_ports[(dpid, port_no)] = True
```

```
120                         else:
121                             self.logger.info("****** TRAFFICO INTERMEDIO - POSSIBILE ATTACCO ******")
122                     else:
123                         self.logger.info("****** PORTA GIÀ BLOCCATA ******")
124                 else:
125                     self.alarm[dpid, port_no] = False
126                     self.logger.info("----------NON SUPERO LA SOGLIA----------")
127 
128                     if (dpid, port_no) in self.blocked_ports:
129                         self.blocked_ports.remove((dpid, port_no))
130                         if (dpid, port_no) in self.entry_ports:
131                             del self.entry_ports[(dpid, port_no)]
132 
133                 self.logger.info(
134                     "Switch: %s, Port: %s, Rx Throughput: %d bps, Tx Throughput: %d bps, Alarm: %s",
135                     dpid, port_no, rx_throughput, tx_throughput, self.alarm[dpid, port_no]
136                 )
137                 self.logger.info("\n\n")
138             else:
139                 self.prev_stats[(dpid, port_no)] = (current_rx_bytes, current_tx_bytes)
```

Osservazioni:

- *def port_stats_reply_handler* → questa funzione gestisce le risposte contenenti le statistiche delle porte (OFPPortStatsReply). In particolare:
    - Ottiene il corpo del messaggio della risposta (**body**) contenente le statistiche delle porte, nonché l'ID del datapath (**dpid**).
    - Analizza le statistiche delle porte per ogni switch (**datapath**) e calcola il throughput di ricezione (*rx*) e trasmissione (*tx*) in bps (moltiplicazione per 8 per passare da byte a bit e divisione per l'intervallo in modo da ottenere la velocità in bps).
    - Se il throughput di ricezione supera la soglia fissata inizialmente (**self.threshold**), attiva un allarme (**self.alarm[dpid, port_no] = True**) e possiamo gestire il traffico su quella specifica porta.
    - Si verifica se la porta è già bloccata. Se non lo è, ovvero la coppia **(dpid, port_no)** appartiene a **self.blocked_ports**, allora verifica se è la prima volta che si rileva un throughput maggiore della soglia.
    - Se è effettivamente la prima volta, allora chiama la funzione di **check_and_block_port** per capire chi è stato a generare quel traffico, facendo uso del contenuto del file json.
    - Altrimenti, se la rilevazione era già stata effettuata, si considera il traffico come intermedio e, dunque, potenzialmente pericoloso.
    - Quando la soglia non viene superata, l'allarme viene settato a False (**self.alarm[dpid, port_no] = False**), controlliamo se la porta era bloccata e rimuove il blocco se necessario.
    - Indipendentemente dal valore del traffico, vengono stampate le informazioni le statistiche.

N.B. la logica implementata in questa funzione è risultata essere fondamentale per gestire correttamente il traffico malevolo.

```python
    def check_and_block_port(self, datapath, port_no):
        dpid = datapath.id

        if (dpid, port_no) in self.host_info:
            self.logger.info("Traffico generato direttamente dall'host")
            self.logger.info("****** CHIAMO LA FUNZIONE PER BLOCCARE IL TRAFFICO ******")
            self.block_port(datapath, port_no, self.block_duration)
            self.blocked_ports.add((dpid, port_no))
        else:
            self.logger.info("Traffico intermedio - possibile attacco")
            if (dpid, port_no) in self.entry_ports:
                del self.entry_ports[(dpid, port_no)]

    def block_port(self, datapath, port_no, duration):
        ofproto = datapath.ofproto
        parser = datapath.ofproto_parser

        match = parser.OFPMatch(in_port=port_no)
        actions = []

        self.add_flow(datapath, 2, match, actions)

        # accesso alle chiavi mac-ip
        attacker_info = self.host_info.get((datapath.id, port_no), {})
        attacker_mac = attacker_info.get('mac', 'unknown')
        attacker_ip = attacker_info.get('ip', 'unknown')

        # stampa delle informazioni sulla coppia switch-porta
        self.logger.info("Blocco del traffico sulla porta %s dello switch %s per %s secondi", port_no, datapath.id, duration)

        # stampa dell'indirizzo MAC e IP dell'host attaccante
        self.logger.info("MAC attaccante: %s, IP attaccante: %s", attacker_mac, attacker_ip)

        # Avvia un thread per sbloccare la porta dopo una durata specifica
        threading.Thread(target=self.unblock_port_after_duration, args=(datapath, port_no, duration)).start()
```

Osservazioni:

- *def check_and_block_port* → funzione che verifica se il traffico su una porta specifica di uno switch è generato direttamente da un host o se è traffico intermedio (potenzialmente accumulatosi dalla generazione di diversi host) e decide se bloccare la porta o meno. Il controllo viene effettuato utilizzando il contenuto del dizionario **self.host_info**, caricato con i dati del file json (come specificato precedentemente). Nel momento in cui la coppia **(dpid, port_no)** è presente in tale dizionario, viene chiamata la funzione effettiva di blocco della porta (**block_port**) e la coppia viene aggiunta al dizionario di porte bloccate (**self.blocked_ports**).

- *def block_port* → funzione che blocca il traffico su una specifica porta di uno switch per una durata specifica (**duration**).
    - Viene creata una nuova corrispondenza di pacchetti (**match**) per quelli in entrata sulla porta specificata (**port_no**).
    - Viene creata una lista vuota di azioni (**actions = [ ]**) → significa che pacchetti corrispondenti non verranno inoltrati da nessuna parte, quindi saranno effettivamente bloccati.
    - Viene aggiunta una regola di flusso al datapath con priorità 2, usando il match e le azioni specificate.
    - Successivamente, la funzione preleva delle informazioni dal dizionario **self.host_info**, ovvero gli indirizzi mac e ip specificati (**attacker_mac**, **attacker_ip**), utilizzando come chiave la coppia (dpid, port_no). Se non ci sono informazioni disponibili, restituisce **unknown**.
    - Infine, la funzione stampa le informazioni sull'attaccante e sull'operazione di blocco, per poi lanciare un thread che si occuperà dello sblocco (**unblock_port_after_duration**).

N.B. poiché il thread viene avviato immediatamente con *start(),* la funzione "unblock_port_after_duration" verrà eseguita in parallelo, consentendo di sbloccare la porta automaticamente dopo la durata specificata.

```
223     def unblock_port_after_duration(self, datapath, port_no, duration):
224         time.sleep(duration)
225 
226         ofproto = datapath.ofproto
227         parser = datapath.ofproto_parser
228 
229         match = parser.OFPMatch(in_port=port_no)
230         actions = [parser.OFPActionOutput(ofproto.OFPP_NORMAL)]
231 
232         self.add_flow(datapath, 2, match, actions)
233 
234         self.logger.info("Sblocco del traffico sulla porta %s dello switch %s", port_no, datapath.id)
235         self.blocked_ports.discard((datapath.id, port_no))
236         if (datapath.id, port_no) in self.entry_ports:
237             del self.entry_ports[(datapath.id, port_no)]
```

Osservazioni:

- *def unblock_port_after_duration* → questa funzione si occupa di sbloccare una porta di uno switch dopo una determinata durata (**time.sleep(duration)**).
    - Dopo aver atteso **duration** (in secondi), invia una nuova regola di flusso per sbloccare la porta specificata.
    - Viene creata una lista di azioni con un singolo elemento, il quale indica che i pacchetti devono essere inoltrati normalmente (**OFPP_NORMAL**).
    - Aggiunge una regola di flusso al datapath con priorità 2, usando il match e le azioni specificate. Poiché l'azione è di inoltro normale, questa regola sblocca il traffico sulla porta specificata.
    - Viene stampato un messaggio informativo, il quale indica che il traffico corrispondente alla coppia (dpid, port_no) è stato correttamente sbloccato.
    - Viene, infine, rimossa tale coppia dal dizionario di porte bloccate (**self.blocked_ports.discard(…)**) e dal dizionario **self.entry_ports** (se presente).

5.1.2 Mitigazione dell'attacco

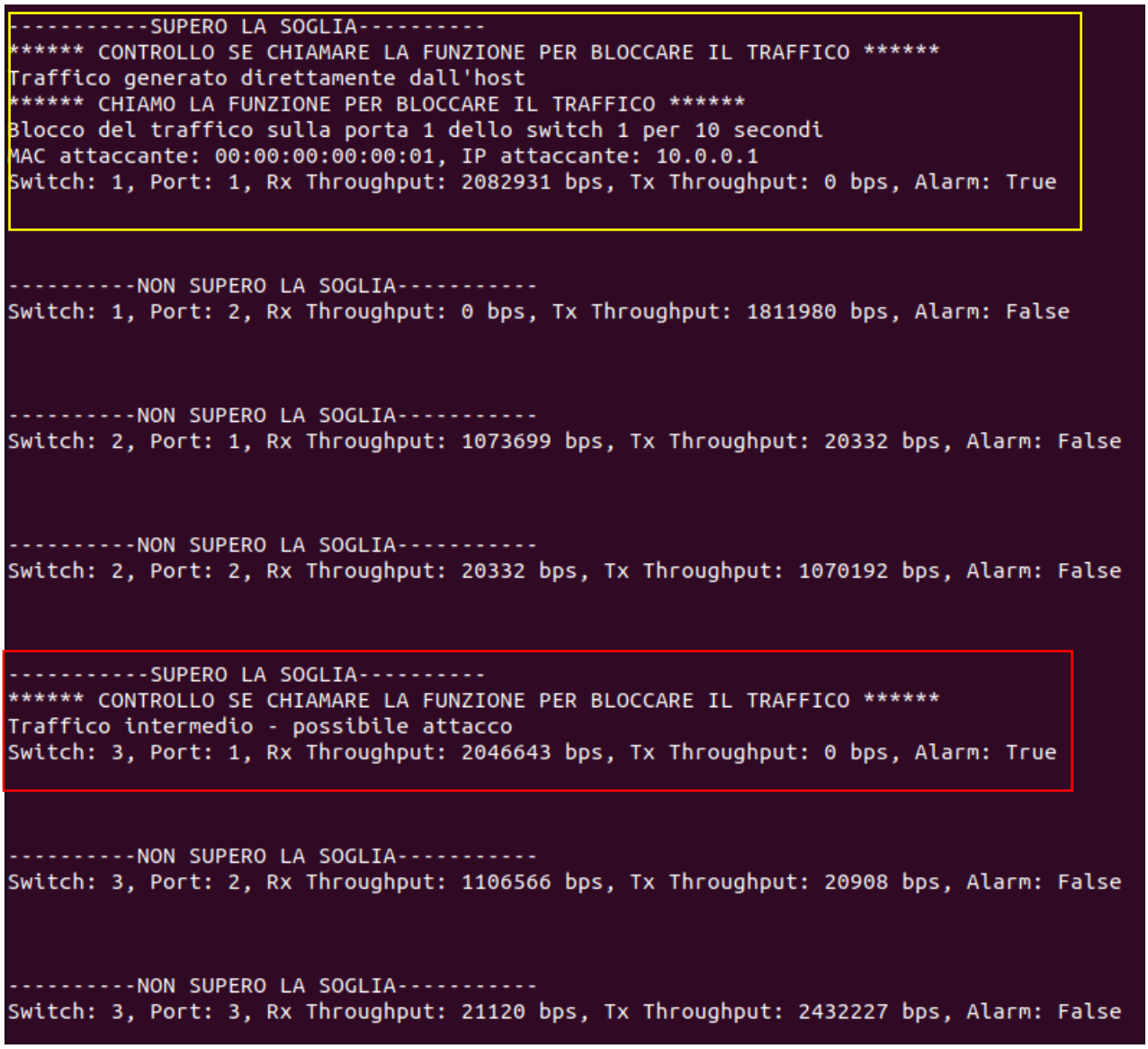
Avendo utilizzato la strategia di mitigazione indicata nel precedente paragrafo, mostriamo ora i risultati ottenuti effettuando la simulazione con **wireshark**:



Com'è possibile osservare dal grafico, l'andamento del traffico TCP di h2 decresce man mano che l'andamento del traffico UDP di h1 (attaccante) aumenta. A circa 10 secondi inizia l'attacco UDP, il quale, superando la soglia impostata, fa attivare il meccanismo di mitigazione. Il blocco durerà 10 secondi, per poi sbloccare la porta su cui h1 manda il proprio traffico. Ciò che succede è che, poiché viene rilevato altre due volte il superamento della soglia, la stessa porta viene bloccata nuovamente.

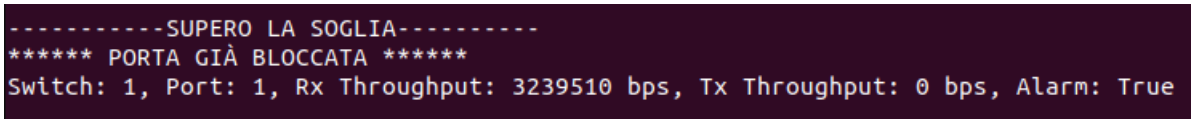
N.B. ogni volta che l'attacco viene bloccato, il traffico TCP riesce ad avere un andamento più stabile.

Di seguito riportiamo le informazioni di log ottenute durante la simulazione:

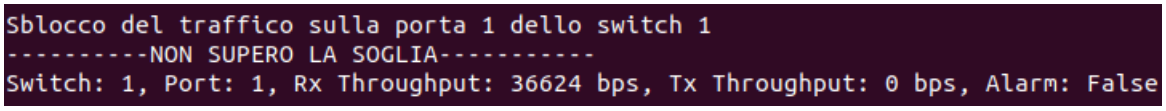


Si nota come, nel momento in cui il traffico che supera la soglia proviene direttamente da un host, vengano fornite le informazioni (indirizzo mac e ip) dell'attaccante, nonché il throughput e il segnale di allarme. Inoltre, come ci aspettavamo, viene anche chiamata correttamente la funzione di blocco, specificando la durata scelta (in giallo).

Viceversa, se il traffico non viene generato direttamente da un host ma è intermedio, viene comunque stampato un messaggio di avvertimento, così da non ignorare totalmente anche traffici potenzialmente rischiosi per la rete (in rosso).



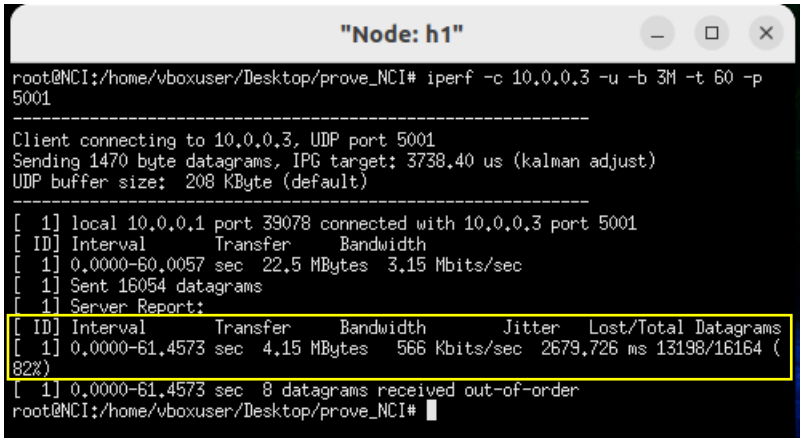
Come ulteriore prova del bloccaggio di una porta, vi è il caso in cui sulla medesima porta arriva nuovamente traffico superiore alla soglia, fornendo la stampa riportata precedentemente.



Infine, nel momento in cui è scaduto il tempo di blocco, viene sbloccata correttamente la porta.

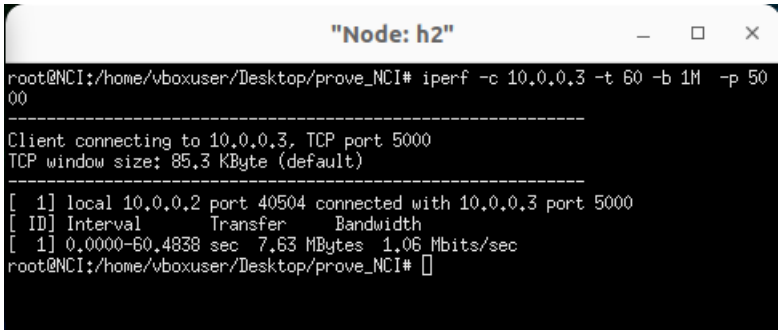
Nella simulazione, è stato generato traffico:

- *UDP da h1 ad h3*



È possibile osservare dal terminale come la percentuale di pacchetti UDP persi sia pari all'82%, indicando che il sistema di difesa messo in atto sta funzionando correttamente nel rilevare e bloccare l'attacco (attacco significativamente mitigato).

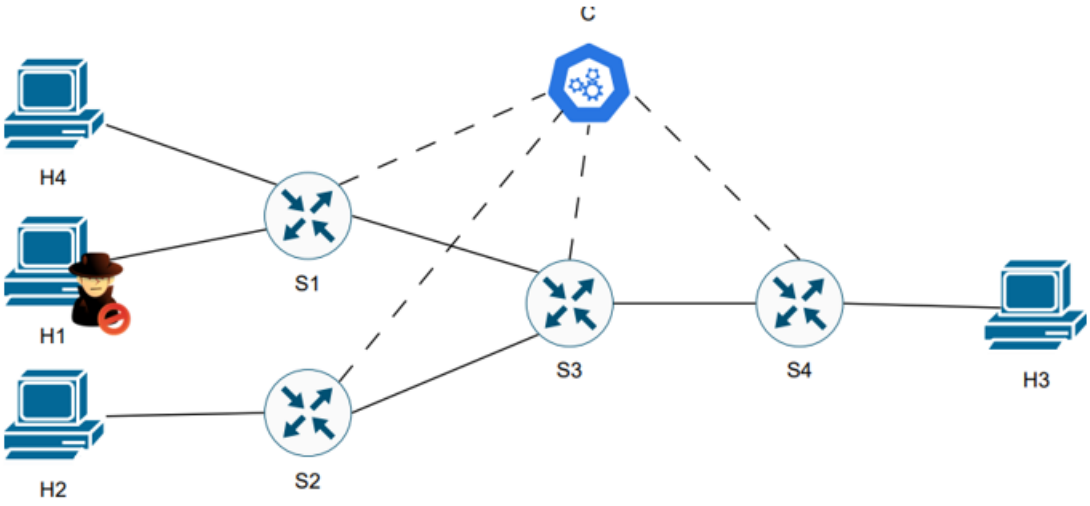
- *TCP da h2 ad h3*



N.B. in entrambe le immagini sono presenti i parametri di trasmissione che sono stati utilizzati (uguali a quelli discussi nei *paragrafi 4.1* e *4.2* in cui veniva effettuato l'attacco senza mettere in atto nessuna strategia di difesa).

5.2 Topologia 2: soluzioni

Esattamente come richiesto dalla traccia, abbiamo esteso la soluzione implementata nei capitoli precedenti anche ad un ulteriore topologia, in cui l'attaccante (host h1) è agganciato ad uno switch sul quale si trovano anche altri utenti (h4, in questo caso). Di seguito, la nuova topologia di riferimento:



5.2.1 Script topologia

Iniziamo col riportare il codice della nuova topologia:

```python
#!/usr/bin/python

# importazione librerie
import random
import threading
import time
from mininet.net import Mininet
from mininet.node import RemoteController, OVSKernelSwitch, Host
from mininet.link import TCLink
from mininet.cli import CLI
from mininet.log import setLogLevel, info

class Environment(object):
    @staticmethod
    def myTopology():
        # crea un'istanza di Mininet per la simulazione della rete
        net = Mininet(controller=RemoteController, link=TCLink, switch=OVSKernelSwitch)

        # aggiuta di un controller alla rete
        info('*** Adding controller\n')
        c0 = net.addController(name='c0', controller=RemoteController)
        c0.start()

        # aggiuta degli host alla rete
        info('*** Adding hosts\n')
        H1 = net.addHost('h1', cls=Host, mac='00:00:00:00:00:01', ip='10.0.0.1')
        H2 = net.addHost('h2', cls=Host, mac='00:00:00:00:00:02', ip='10.0.0.2')
        H3 = net.addHost('h3', cls=Host, mac='00:00:00:00:00:03', ip='10.0.0.3')
        H4 = net.addHost('h4', cls=Host, mac='00:00:00:00:00:04', ip='10.0.0.4')

        # aggiuta degli switch alla rete
        info('*** Adding switches\n')
        S1 = net.addSwitch('s1', cls=OVSKernelSwitch, protocols='OpenFlow13')
        S2 = net.addSwitch('s2', cls=OVSKernelSwitch, protocols='OpenFlow13')
        S3 = net.addSwitch('s3', cls=OVSKernelSwitch, protocols='OpenFlow13')
        S4 = net.addSwitch('s4', cls=OVSKernelSwitch, protocols='OpenFlow13')
```

```
37
38          # aggiuta dei link tra gli host e gli switch alla rete
39          info('*** Creating links\n')
40          net.addLink(H1, S1, bw=6, delay='0.0025ms')
41          net.addLink(H4, S1, bw=6, delay='0.0025ms')
42          net.addLink(H2, S2, bw=6, delay='0.0025ms')
43          net.addLink(S1, S3, bw=10, delay='25ms')
44          net.addLink(S2, S3, bw=10, delay='25ms')
45          net.addLink(S3, S4, bw=10, delay='25ms')
46          net.addLink(S4, H3, bw=6, delay='0.0025ms')
47
48          info('*** Starting network\n')
49          net.build()
50          net.start()
51
52          info('*** Testing network connectivity\n')
53          net.pingAll()
54
55          info('*** Running CLI\n')
56          CLI(net)
57
58          info('*** Stopping network\n')
59          net.stop()
60
61  # avvio dell'ambiente di rete e la CLI di Mininet
62  if __name__ == '__main__':
63      setLogLevel('info')
64      Environment.myTopology()
```

In questo caso, le modifiche apportate alla topologia precedente sono:

- aggiunta dell'host h4
- aggiunta del link che collega il nuovo host (h4) allo switch 1
- modifica della banda associata ai link che collegano gli switch tra loro (da 3 a 10) → tale scelta è stata fatta poiché si è supposto che, avendo più host, fosse necessario disporre di link "intermedi" nella rete con una capacità di banda maggiore (così da supportare traffici più elevati)

5.2.2 Soluzione 1 – Estensione alla nuova topologia

Con la nuova topologia in nostro possesso, l'obiettivo è quello di non penalizzare anche altri host collegati allo stesso switch dell'attaccante; in quanto un blocco non implementato correttamente potrebbe bloccare tutto il traffico proveniente dallo switch sul quale giace l'attaccante e di conseguenza bloccando anche tutti gli utenti che passano per il medesimo switch. In questo caso la soluzione precedente è sempre valida perché il controller blocca solo ed esclusivamente la porta d'ingresso dell'host che genera traffico rilevato come malevolo, lasciando passare tutti il traffico degli host benevoli.

Prima di procedere, dato che la topologia è cambiata, è necessario aggiornare opportunamente il file *json*, aggiungendo il nuovo host h4, specificando lo switch al quale è collegato con la relativa porta, indirizzo mac "**00:00:00:00:00:04**" ed indirizzo ip "**10.0.0.4**".

Open | *host_info_net2.json ~/Desktop | Save

```
1 {
2     "(1, 1)": {"mac": "00:00:00:00:00:01", "ip": "10.0.0.1"},
3     "(2, 1)": {"mac": "00:00:00:00:00:02", "ip": "10.0.0.2"},
4     "(1, 2)": {"mac": "00:00:00:00:00:04", "ip": "10.0.0.4"}
5 }
```

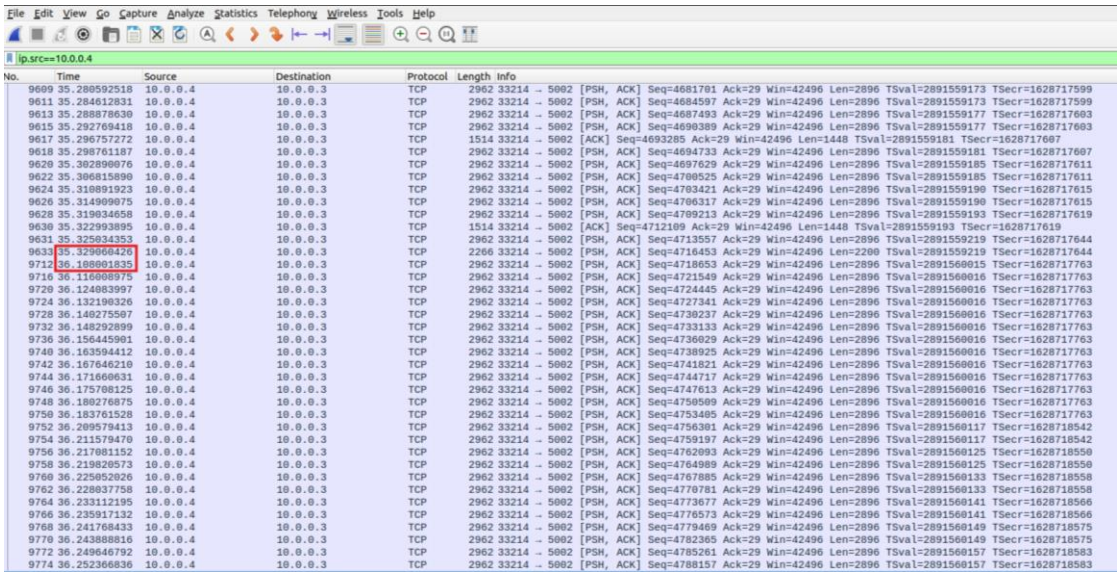
Ora, è possibile procedere con la generazione del traffico per gli host precedenti, aggiungendo anche l'host h4 che genererà traffico TCP a 1 Mbps. Nel primo esempio, mostrato di seguito, è stato impostato il tempo di blocco porta pari a 10 secondi. È possibile notare che il traffico UDP viene bloccato, lasciando passare solo il traffico TCP.

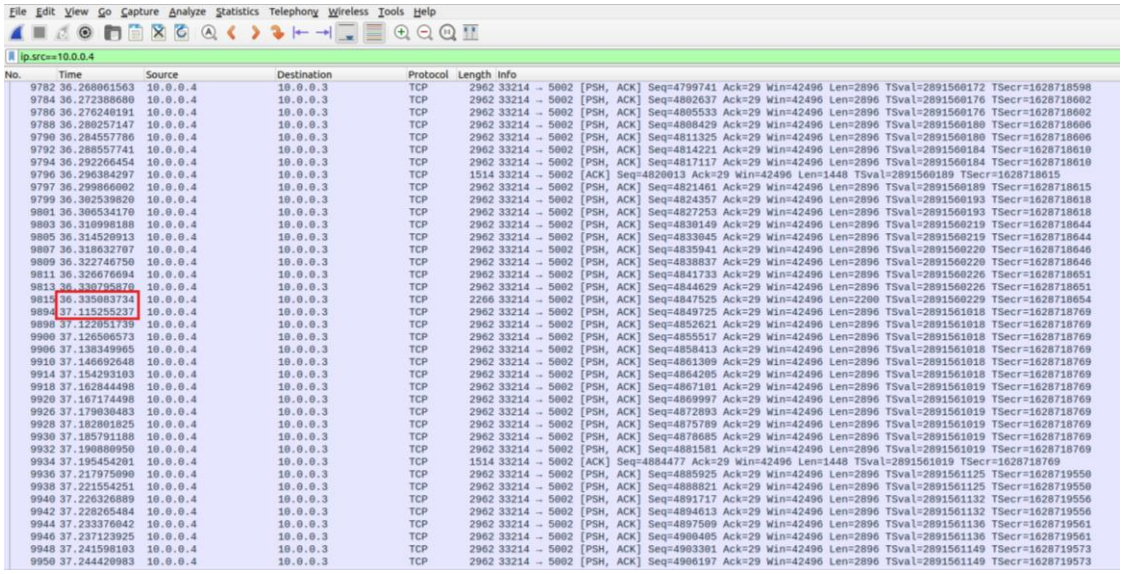


Con un intervallo di 7 secondi gli eventi di blocco e sblocco avranno una frequenza maggiore, dato che l'intervallo è più breve. Un intervallo minore potrebbe essere utile per fornire una maggiore dinamicità al controller, evitando di lasciare bloccata la porta per un periodo di tempo troppo lungo, tuttavia, di contro, in questo modo aumenta l'overhead per la gestione dei ripetuti eventi di stati della connessione. Di seguito vi è indicato il grafico corrispondente:



Dato che l'**obiettivo** era quello di non bloccare il traffico proveniente da h4, facendo uso del software wireshark, è possibile sincerarsi che tutti i pacchetti generati dall'host 10.0.0.4 arrivino a destinazione verso 10.0.0.3, infatti è possibile vedere come i pacchetti vengano trasmessi durante tutta la durata di generazione del traffico (60 secondi) senza subire alcuna interruzione. Ovviamente non è possibile rappresentare tutta la sequenza di pacchetti nell'intervallo di 60 secondi, però di seguito è mostrato un esempio di continuità dei pacchetti avendo impostato il filtro **ip.src==10.0.0.4** sul software. È possibile verificare quanto detto con il file di cattura in nostro possesso (esportato in formato *csv* per una maggiore flessibilità).





5.2.3 Soluzione 1 – Introduzione alla generazione di traffico variabile

Finora abbiamo lavorato con l'assunzione che l'host h1 generasse esclusivamente traffico UDP malevolo in modo **costante**. Ciò non basta per verificare la robustezza del controllore, in quanto una tecnica di elusione di un controller troppo basilare, potrebbe basarsi sulla generazione di traffico intermittente, quindi generando prima ad un rate tale da manifestare un vero e proprio attacco DoS, per poi passare alla generazione di un traffico ad un normale livello di rate, simile a quello generato da utenti legittimi cercando di eludere i controller, per poi riprendere con la generazione con un throughput troppo elevato.

Al fine di realizzare questo comportamento è stato implementato uno *script bash*, facendo in modo che il terminale di h3 generasse traffico UDP prima a 4Mbps, poi 1Mbps ed infine 6Mbps, tutti per un intervallo di tempo di 20 secondi.

Open *bitrate_var.sh ~/Desktop Save

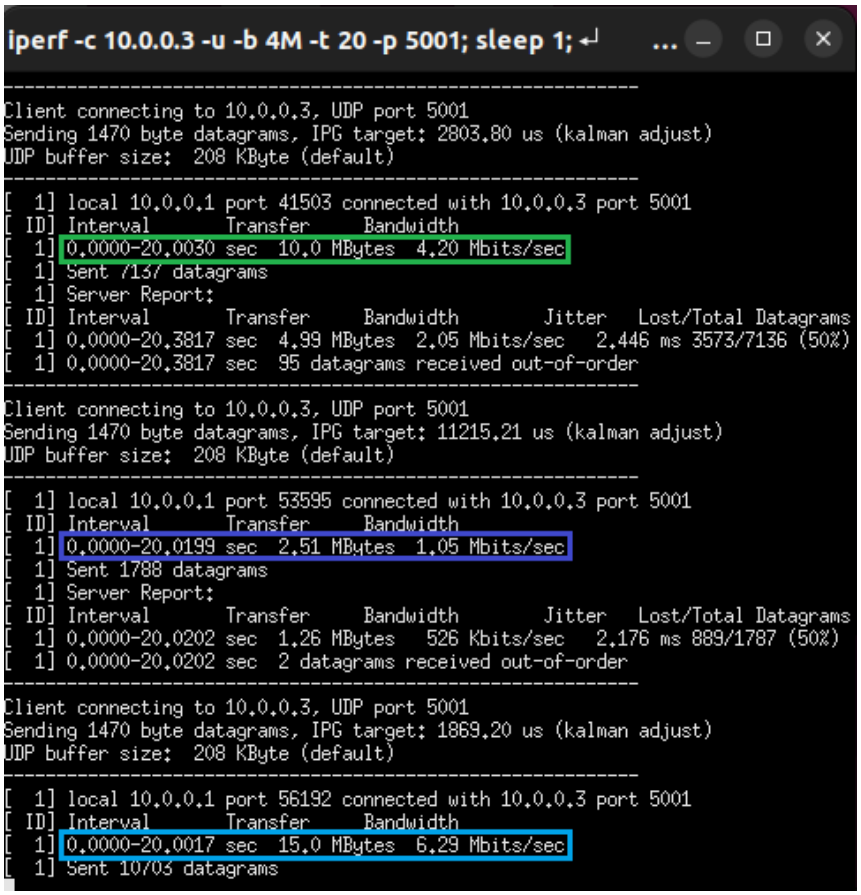
*host_info_net2.json *bitrate_var.sh

```
1 #!/bin/bash
2
3 # Genera traffico con bitrate variabile
4 xterm -e "iperf -c 10.0.0.3 -u -b 4M -t 20 -p 5001; sleep 1;
5           iperf -c 10.0.0.3 -u -b 1M -t 20 -p 5001; sleep 1;
6           iperf -c 10.0.0.3 -u -b 6M -t 20 -p 5001"
```

Per l'esecuzione dello script è necessario fornire i permessi di esecuzione a quest'ultimo tramite il comando "**chmod +x nome_script.sh**" nel terminale in cui è contenuto il file. Dopodiché, avviamo i terminali xterm per ogni host, e al primo host dobbiamo far eseguire il comando "**./bitrate_vr.sh**".



Una volta avviato lo script, si aprirà man mano che le precedenti generazioni terminano, i vari comandi eseguiti in ordine, si evince dall'immagine che tutte e tre le generazioni durano un totale di 20 secondi rispettivamente a circa 4Mbits/sec, 1Mbits/sec e 6Mbits/sec.



Analizzando il grafico generato da wireshark è possibile notare come il traffico venga bloccato nella prima generazione a *4Mbits/sec*. Successivamente, il traffico passa tranquillamente quando la generazione è ridotta a *1Mbits/sec*, poiché a questa velocità non influisce sul passaggio del traffico TCP. Infine, viene nuovamente bloccato quando la generazione del traffico aumenta arrivando a *6Mbits/sec*.



5.2.4 Soluzione 2 – Approccio Dinamico

La soluzione adottata finora non rappresenta un approccio completamente dinamico. Anche se era possibile ridurre l'intervallo di blocco (*paragrafo 5.2.2*), è possibile migliorare il controllo implementando una logica più sofisticata. Proponiamo di sbloccare la porta solo se, per una durata prestabilita in secondi, che possiamo definire, il traffico generato dall'attaccante non supera mai la soglia di threshold. Pertanto, il criterio di sblocco cambia: *la porta verrà sbloccata solo se, per il periodo di tempo specificato, il traffico non supera mai la soglia determinata*.

Di seguito sono mostrati i cambiamenti apportati al codice, al fine di implementare il suddetto cambiamento:

```
11  class SimpleSwitch13(app_manager.RyuApp):
12      OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]
13
14      def __init__(self, *args, **kwargs):
15          super(SimpleSwitch13, self).__init__(*args, **kwargs)
16          self.mac_to_port = {}
17          self.datapaths = {}
18          self.stats_interval = 5  # Intervallo per raccogliere le statistiche dei porti (in secondi)
19          self.threshold = 2 * (10**6)  # Soglia in bit per secondo (2 Mbps)
20          self.alarm = {}
21          self.blocked_ports = set()
22          self.entry_ports = {}  # Dizionario per tenere traccia delle porte di ingresso per ogni flusso
23          self.block_duration = 10 # Durata del blocco in secondi (prima prova: 5)
24          self.prev_stats = {}
25          self.host_info = {}  # Dizionario per tenere traccia degli host e delle loro porte e switch
26          self.label = set() # Dizionario per le etichette
27
28          self.load_host_info_json("host_info_net2.json")
```

È stato aggiunto un nuovo dizionario, per tenere traccia delle porte che superano la soglia nell'intervallo di tempo impostato, in questo modo riusciamo a capire se per un certo periodo di tempo è stato almeno una volta generato traffico malevolo.

```
102                 if (dpid, port_no) in self.prev_stats:
103                     prev_rx_bytes, prev_tx_bytes = self.prev_stats[(dpid, port_no)]
104                     rx_bytes_diff = current_rx_bytes - prev_rx_bytes
105                     tx_bytes_diff = current_tx_bytes - prev_tx_bytes
106
107                     rx_throughput = (rx_bytes_diff * 8) / self.stats_interval
108                     tx_throughput = (tx_bytes_diff * 8) / self.stats_interval
109
110                     self.prev_stats[(dpid, port_no)] = (current_rx_bytes, current_tx_bytes)
111
112                     if rx_throughput > self.threshold:
113                         self.alarm[dpid, port_no] = True
114                         self.logger.info("----------SUPERO LA SOGLIA----------")
115
116                         if (dpid, port_no) not in self.blocked_ports:
117                             if (dpid, port_no) not in self.entry_ports:
118                                 self.logger.info("****** CONTROLLO SE CHIAMARE LA FUNZIONE PER BLOCCARE IL TRAFFICO ******")
119                                 self.check_and_block_port(datapath, port_no)
120                                 self.entry_ports[(dpid, port_no)] = True
121                             else:
122                                 self.logger.info("****** TRAFFICO INTERMEDIO - POSSIBILE ATTACCO ******")
123                         else:
124                             self.logger.info("****** PORTA GIÀ BLOCCATA ******")
125                             if (dpid, port_no) not in self.label:
126                                 self.logger.info("AGGIUNTA ALLA LISTA DI LABEL")
127                                 self.label.add((dpid, port_no)) # coppia (dpid, port_no) aggiunta
128                                 print(self.label) # stampa di debug
129                     else:
130                         self.alarm[dpid, port_no] = False
131                         self.logger.info("----------NON SUPERO LA SOGLIA----------")
132
133                         if (dpid, port_no) in self.blocked_ports:
134                             self.blocked_ports.remove((dpid, port_no))
135                             if (dpid, port_no) in self.entry_ports:
136                                 del self.entry_ports[(dpid, port_no)]
```

Nella funzione *def port_stats_reply_handler* il cambiamento aggiunto risiede nelle righe di codice che vanno da 125 a 128. Nelle quali, per prima cosa, viene controllato se la coppia **(dpid, port_no)** è già presente nel set *self.label*: se la coppia non è già nell'elenco, il codice prosegue. Dopodiché, la coppia **(dpid, port_no)** viene

aggiunta al set *self.label*. Questo significa che ora *self.label* conterrà questa coppia e non la aggiungerà di nuovo se già esistente, grazie alla natura dei set che non permettono duplicati.

```
231     def unblock_port_after_duration(self, datapath, port_no, duration):
232         while True:
233             time.sleep(duration)
234 
235             if (datapath.id, port_no) not in self.label:
236                 ofproto = datapath.ofproto
237                 parser = datapath.ofproto_parser
238 
239                 match = parser.OFPMatch(in_port=port_no)
240                 actions = [parser.OFPActionOutput(ofproto.OFPP_NORMAL)]
241 
242                 self.add_flow(datapath, 2, match, actions)
243 
244                 self.logger.info("Sblocco del traffico sulla porta %s dello switch %s", port_no, datapath.id)
245                 self.blocked_ports.discard((datapath.id, port_no))
246                 if (datapath.id, port_no) in self.entry_ports:
247                     del self.entry_ports[(datapath.id, port_no)]
248                 break
249             else:
250                 self.logger.info("Rimozione dalla lista della coppia (%s, %s)", port_no, datapath.id)
251                 self.label.remove((datapath.id, port_no)) # rimozione della coppia (dpid, port_no) dalla lista di etichette
252                 print(self.label) # stampa di debug
```

La funzione di sblocco cambia drasticamente, il nuovo funzionamento è il seguente:

- La funzione inizia con un ciclo infinito (**while True**). Questo ciclo è stato progettato per continuare fino a quando non viene interrotto manualmente. All'interno di questo ciclo, la funzione aspetta per il numero di secondi specificato dal parametro duration. Durante questo periodo di attesa, non fa nulla altro che dormire.
- Dopo l'attesa, la funzione controlla se la porta che stiamo cercando di sbloccare è ancora nella lista delle porte bloccate, rappresentata da *self.label*. Se la coppia che identifica la porta **(datapath.id, port_no)** non è più nella lista *self.label*, allora significa che è giunto il momento di sbloccare la porta.
- A questo punto, la funzione crea una nuova regola di flusso (**add_flow**) per lo switch specificando che il traffico deve essere gestito normalmente (**OFPP_NORMAL**).
- Poi vengono aggiornate le liste *self.blocked_ports* e *self.entry_ports* per tenere traccia del fatto che la porta non è più bloccata. Se la porta era nella lista delle porte di ingresso, viene rimossa.
- Concludendo le operazioni di sblocco e pulizia, la funzione interrompe il ciclo infinito con un *break*, poiché l'obiettivo è stato raggiunto e non è più necessario continuare a monitorare la porta.
- Viceversa, se la coppia che identifica la porta **(datapath.id, port_no)** è ancora nella lista *self.label*, allora viene effettuata la **rimozione** di tale coppia dal set, così da resettare il conteggio relativo a tale coppia.

5.2.5 Soluzione 2 – Esecuzione con traffico costante

In questo caso, il nuovo controllo sblocca la porta - sulla quale viene rilevato l'attacco DoS - solo se, dopo l'intervallo di tempo preimpostato, non viene superata la soglia massima. Il grafico del traffico generato è il seguente, dove:

- La *curva rossa* rappresenta il *traffico UDP*, caratterizzato da vari picchi nei quali il controller effettua l'analisi delle statistiche di rete, per poi tendere a zero una volta che l'analisi ha rilevato il possibile attacco e procede con il blocco della porta sulla quale viene generato il traffico malevolo.
- La *curva verde* rappresenta il *traffico TCP* che solo dopo una piccola variazione iniziale, tende a rimanere costante grazie alla mitigazione dell'attacco.



Il controller inizia a bloccare la porta 1 dello switch 1, perché rileva l'attacco, dato che il threshold raggiunge un valore maggiore della nostra soglia.

```
-----------SUPERO LA SOGLIA----------
****** CONTROLLO SE CHIAMARE LA FUNZIONE PER BLOCCARE IL TRAFFICO ******
Traffico generato direttamente dall'host
****** CHIAMO LA FUNZIONE PER BLOCCARE IL TRAFFICO ******
Blocco del traffico sulla porta 1 dello switch 1 per 10 secondi
MAC attaccante: 00:00:00:00:00:01, IP attaccante: 10.0.0.1
Switch: 1, Port: 1, Rx Throughput: 3161961 bps, Tx Throughput: 291 bps, Alarm: True
```

Vengono correttamente rilevati i traffici intermedi, causati dal precedente traffico malevolo, ma essendo solo switch che si trovano lungo il percorso nessuna porta viene bloccata.

```
----------NON SUPERO LA SOGLIA-----------
Switch: 1, Port: 2, Rx Throughput: 1073804 bps, Tx Throughput: 23395 bps, Alarm: False



----------NON SUPERO LA SOGLIA-----------
Switch: 1, Port: 3, Rx Throughput: 23395 bps, Tx Throughput: 3256041 bps, Alarm: False



----------NON SUPERO LA SOGLIA-----------
Switch: 2, Port: 1, Rx Throughput: 1056080 bps, Tx Throughput: 20227 bps, Alarm: False



----------NON SUPERO LA SOGLIA-----------
Switch: 2, Port: 2, Rx Throughput: 20227 bps, Tx Throughput: 1056080 bps, Alarm: False



-----------SUPERO LA SOGLIA----------
****** CONTROLLO SE CHIAMARE LA FUNZIONE PER BLOCCARE IL TRAFFICO ******
Traffico intermedio - possibile attacco
Switch: 3, Port: 1, Rx Throughput: 3319942 bps, Tx Throughput: 24121 bps, Alarm: True



----------NON SUPERO LA SOGLIA-----------
Switch: 3, Port: 2, Rx Throughput: 1073923 bps, Tx Throughput: 20966 bps, Alarm: False



----------NON SUPERO LA SOGLIA-----------
Switch: 3, Port: 3, Rx Throughput: 45254 bps, Tx Throughput: 4349904 bps, Alarm: False



-----------SUPERO LA SOGLIA----------
****** CONTROLLO SE CHIAMARE LA FUNZIONE PER BLOCCARE IL TRAFFICO ******
Traffico intermedio - possibile attacco
Switch: 4, Port: 1, Rx Throughput: 4403344 bps, Tx Throughput: 45676 bps, Alarm: True
```

Viene rilevato un traffico generato da h1 ancora superiore alla soglia, il quale porta all'aggiunta della coppia (1, 1) alla lista di etichette:

```
-----------SUPERO LA SOGLIA----------
****** PORTA GIÀ BLOCCATA ******
AGGIUNTA ALLA LISTA DI LABEL
{(1, 1)}
Switch: 1, Port: 1, Rx Throughput: 3232051 bps, Tx Throughput: 224 bps, Alarm: True
```

Poiché la soglia viene costantemente superata, al termine della durata impostata (10 secondi), la soglia viene nuovamente superata, portando alla condizione in cui la coppia viene rimossa ma successivamente aggiunta alla lista:

```
Rimozione dalla lista della coppia (1, 1)
set()
----------SUPERO LA SOGLIA----------
****** PORTA GIÀ BLOCCATA ******
AGGIUNTA ALLA LISTA DI LABEL
{(1, 1)}
Switch: 1, Port: 1, Rx Throughput: 2644297 bps, Tx Throughput: 684 bps, Alarm: True
```

Successivamente, il traffico generato da h1 si mantiene ad un valore leggermente inferiore alla soglia per la durata necessaria a portare allo sblocco della porta 1 switch 1:

```
----------NON SUPERO LA SOGLIA-----------
Switch: 1, Port: 1, Rx Throughput: 1887312 bps, Tx Throughput: 0 bps, Alarm: False
```

```
Sblocco del traffico sulla porta 1 dello switch 1
----------NON SUPERO LA SOGLIA-----------
Switch: 1, Port: 1, Rx Throughput: 336 bps, Tx Throughput: 0 bps, Alarm: False
```

Verso la fine, a ~55 secondi, viene nuovamente chiamata la funzione di blocco in quanto c'è un ulteriore superamento della soglia:

```
----------SUPERO LA SOGLIA----------
****** CONTROLLO SE CHIAMARE LA FUNZIONE PER BLOCCARE IL TRAFFICO ******
Traffico generato direttamente dall'host
****** CHIAMO LA FUNZIONE PER BLOCCARE IL TRAFFICO ******
Blocco del traffico sulla porta 1 dello switch 1 per 10 secondi
MAC attaccante: 00:00:00:00:00:01, IP attaccante: 10.0.0.1
Switch: 1, Port: 1, Rx Throughput: 3050678 bps, Tx Throughput: 67 bps, Alarm: True
```

Dopo l'ultima volta che la coppia (1, 1) viene rimossa dalla lista, non vi è più alcuna generazione di traffico UDP, dunque, la porta verrà sbloccata al termine della durata (10 secondi) che è stata impostata:

```
Rimozione dalla lista della coppia (1, 1)
set()
----------NON SUPERO LA SOGLIA-----------
Switch: 1, Port: 1, Rx Throughput: 0 bps, Tx Throughput: 0 bps, Alarm: False
```

```
Sblocco del traffico sulla porta 1 dello switch 1
----------NON SUPERO LA SOGLIA-----------
Switch: 1, Port: 1, Rx Throughput: 0 bps, Tx Throughput: 0 bps, Alarm: False
```

5.2.6 Soluzione 2 – Esecuzione con traffico variabile

Più interessante è il funzionamento con la generazione del **traffico variabile**. La curva rossa come sempre rappresenta il traffico UDP, caratterizzati da picchi iniziali elevati che procedono a tendere verso lo zero dopo il corretto blocco della porta. Grazie alla mitigazione si può vedere come la curva verde, ovvero il traffico TCP, riesce ad essere molto stabile, grazie alla corretta mitigazione dell'attacco in corso. Come si evince dalla figura mostrata di seguito, la porta viene correttamente sbloccata quando la generazione diminuisce ad 1 Mbits/s, per poi essere bloccata nuovamente quando raggiunge i 6Mbits/s.

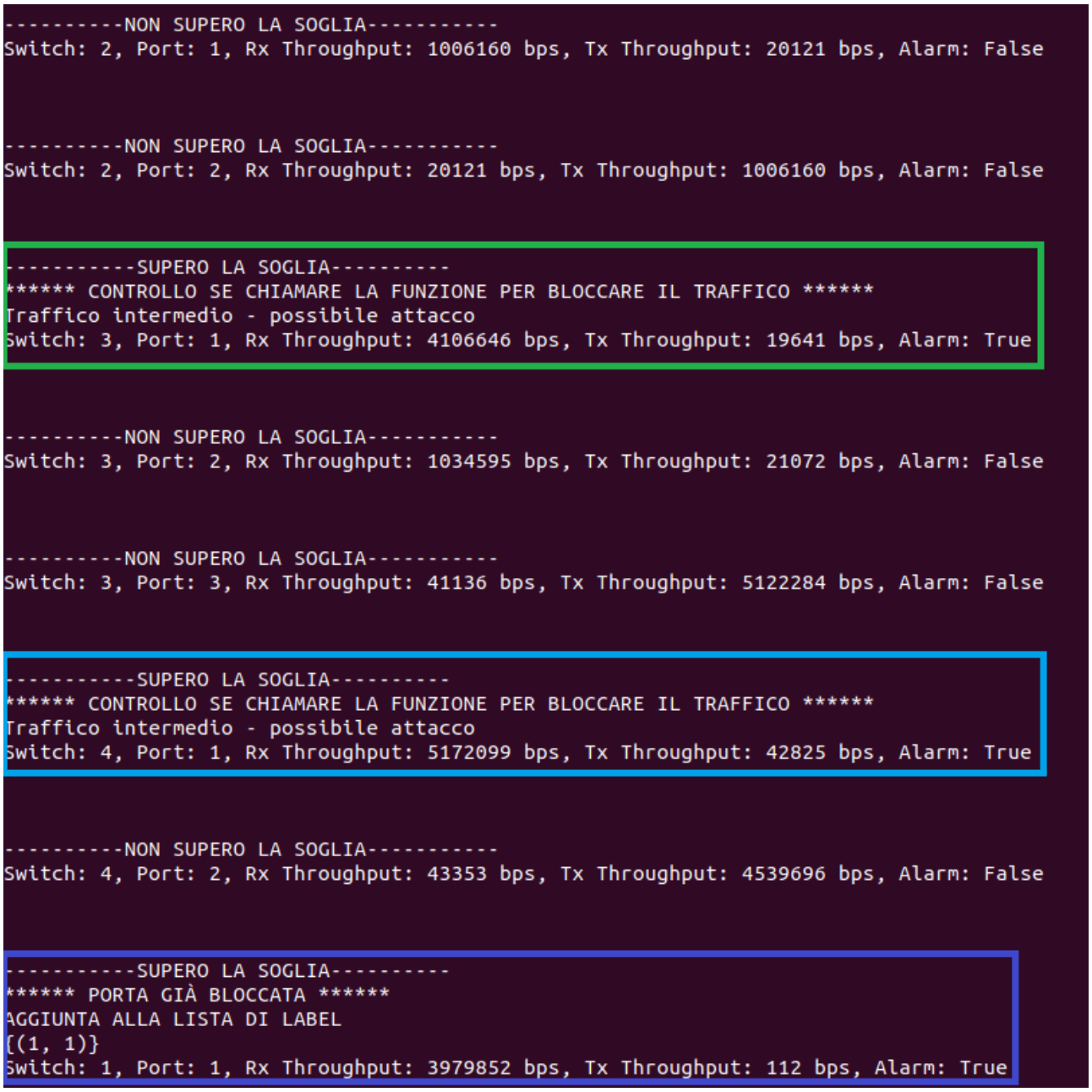


Dalle stampe del controller è possibile verificare che il comportamento sia quello atteso. All'inizio viene rilevato il traffico con throughput troppo elevato e dopo aver collezionato le statistiche di rete, viene bloccata la porta.

```
----------SUPERO LA SOGLIA----------
****** CONTROLLO SE CHIAMARE LA FUNZIONE PER BLOCCARE IL TRAFFICO ******
Traffico generato direttamente dall'host
****** CHIAMO LA FUNZIONE PER BLOCCARE IL TRAFFICO ******
Blocco del traffico sulla porta 1 dello switch 1 per 10 secondi
MAC attaccante: 00:00:00:00:00:01, IP attaccante: 10.0.0.1
Switch: 1, Port: 1, Rx Throughput: 3031257 bps, Tx Throughput: 0 bps, Alarm: True


----------NON SUPERO LA SOGLIA----------
Switch: 1, Port: 2, Rx Throughput: 1072969 bps, Tx Throughput: 19641 bps, Alarm: False
```

Proseguendo viene rilevato il traffico intermedio presente sullo switch 3 porta 1 e sullo switch 4 porta 1, dopo il primo ciclo di controllo, viene rilevato ancora il traffico malevolo sulla porta 1 del primo switch; quindi, vediamo correttamente la stampa "**PORTA GIA' BLOCCATA**" e "**AGGIUNTA ALLA LISTA DI LABEL**", inoltre vediamo anche che il set è correttamente costituito dalla coppia (1,1).



Trascorsi i 10 secondi, che corrispondono alla durata del controllo, la coppia (1,1) viene rimossa dal set; infatti, vediamo che, come ci aspettavamo, il set risulta essere completamente vuoto.

```
----------SUPERO LA SOGLIA----------
****** TRAFFICO INTERMEDIO - POSSIBILE ATTACCO ******
Switch: 4, Port: 1, Rx Throughput: 2154521 bps, Tx Throughput: 54384 bps, Alarm: True



----------NON SUPERO LA SOGLIA----------
Switch: 4, Port: 2, Rx Throughput: 53856 bps, Tx Throughput: 2786924 bps, Alarm: False



Rimozione dalla lista della coppia (1, 1)
set()
----------NON SUPERO LA SOGLIA----------
Switch: 1, Port: 1, Rx Throughput: 448 bps, Tx Throughput: 1020 bps, Alarm: False
```

Continuando, arriviamo all'istante in cui il traffico generato si abbassa ad 1Mbits/s. Dunque, il controllo rileva che il traffico non è tale da disturbare le comunicazioni, pertanto viene sbloccata la porta 1 dello switch 1, perché per almeno 10 secondi non ha mai generato traffico superiore alla soglia impostata. Infatti, la generazione del traffico dura ben 20 secondi, che risultano essere addirittura il doppio del tempo di controllo dello sblocco.

```
----------SUPERO LA SOGLIA----------
****** TRAFFICO INTERMEDIO - POSSIBILE ATTACCO ******
Switch: 4, Port: 1, Rx Throughput: 2146064 bps, Tx Throughput: 48172 bps, Alarm: True



----------NON SUPERO LA SOGLIA----------
Switch: 4, Port: 2, Rx Throughput: 48172 bps, Tx Throughput: 2146176 bps, Alarm: False



Sblocco del traffico sulla porta 1 dello switch 1
----------NON SUPERO LA SOGLIA----------
Switch: 1, Port: 1, Rx Throughput: 268 bps, Tx Throughput: 0 bps, Alarm: False

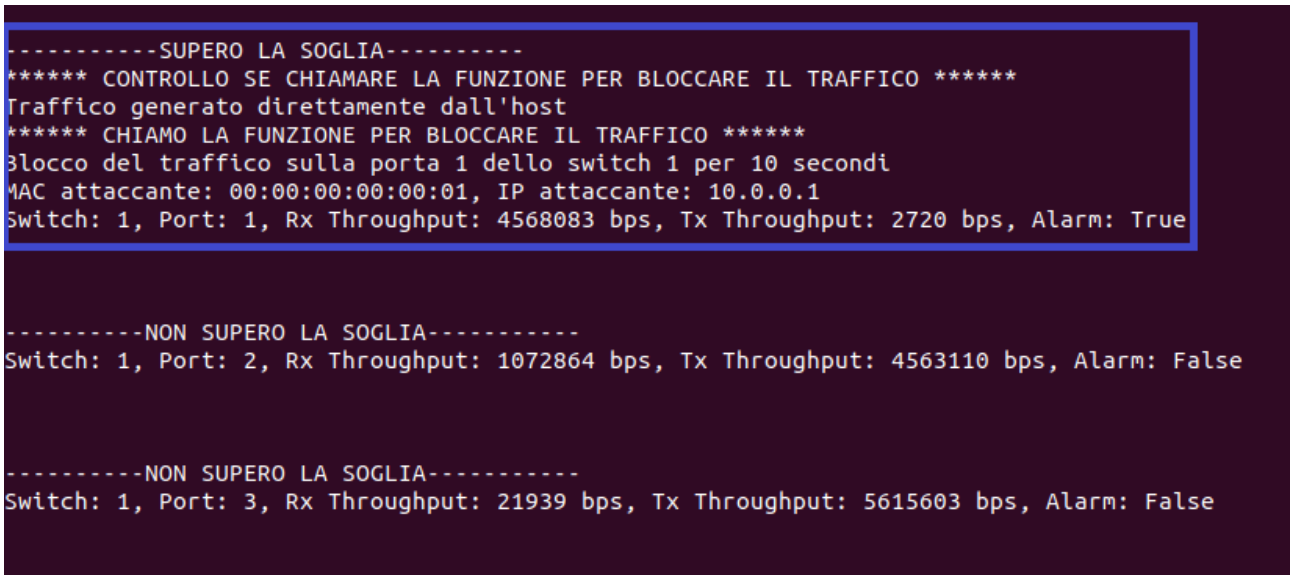


----------NON SUPERO LA SOGLIA----------
Switch: 1, Port: 2, Rx Throughput: 1072864 bps, Tx Throughput: 23865 bps, Alarm: False

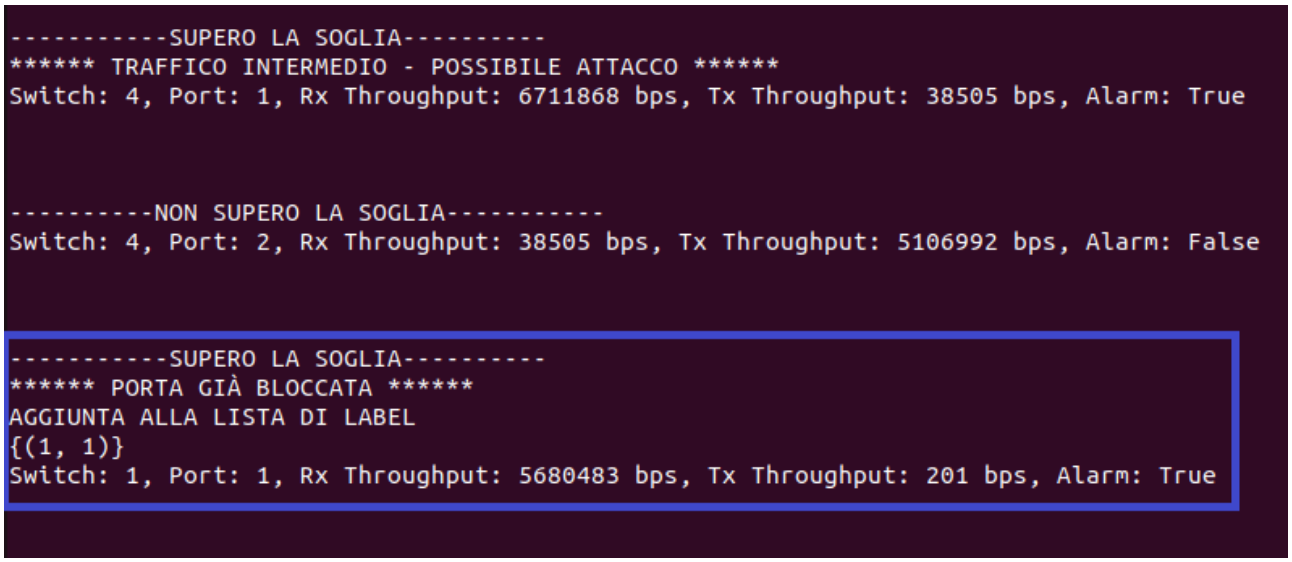


----------NON SUPERO LA SOGLIA----------
Switch: 1, Port: 3, Rx Throughput: 23865 bps, Tx Throughput: 1072864 bps, Alarm: False
```

Quando si arriva alla generazione di 6Mbits/s si procede di nuovo con il blocco della porta 1 sullo switch 1



Dopo il primo ciclo di controllo, si può vedere che la porta supera ancora la soglia, pertanto viene riconosciuto il fatto che quest'ultima risulti già essere bloccata, e viene nuovamente aggiunta la coppia (1,1) al set label, con throughput par a ~ 6Mbits/s.



Trascorrono altri 10 secondi e la coppia viene nuovamente rimossa dal set

```
----------NON SUPERO LA SOGLIA-----------
Switch: 4, Port: 2, Rx Throughput: 58665 bps, Tx Throughput: 3752048 bps, Alarm: False


Rimozione dalla lista della coppia (1, 1)
set()
-----------SUPERO LA SOGLIA----------
****** PORTA GIÀ BLOCCATA ******
AGGIUNTA ALLA LISTA DI LABEL
{(1, 1)}
Switch: 1, Port: 1, Rx Throughput: 5803660 bps, Tx Throughput: 0 bps, Alarm: True


----------NON SUPERO LA SOGLIA-----------
Switch: 1, Port: 2, Rx Throughput: 1073075 bps, Tx Throughput: 24288 bps, Alarm: False


----------NON SUPERO LA SOGLIA-----------
Switch: 1, Port: 3, Rx Throughput: 24288 bps, Tx Throughput: 1073075 bps, Alarm: False


----------NON SUPERO LA SOGLIA-----------
Switch: 2, Port: 1, Rx Throughput: 1072864 bps, Tx Throughput: 24288 bps, Alarm: False
```

Giunti al termine della generazione del traffico possiamo constatare che la porta precedentemente bloccata viene correttamente sbloccata, dato il termine della generazione, con un throughput di 0 bps.

```
----------NON SUPERO LA SOGLIA-----------
Switch: 4, Port: 1, Rx Throughput: 224 bps, Tx Throughput: 0 bps, Alarm: False


----------NON SUPERO LA SOGLIA-----------
Switch: 4, Port: 2, Rx Throughput: 0 bps, Tx Throughput: 336 bps, Alarm: False


Sblocco del traffico sulla porta 1 dello switch 1
----------NON SUPERO LA SOGLIA-----------
Switch: 1, Port: 1, Rx Throughput: 0 bps, Tx Throughput: 0 bps, Alarm: False

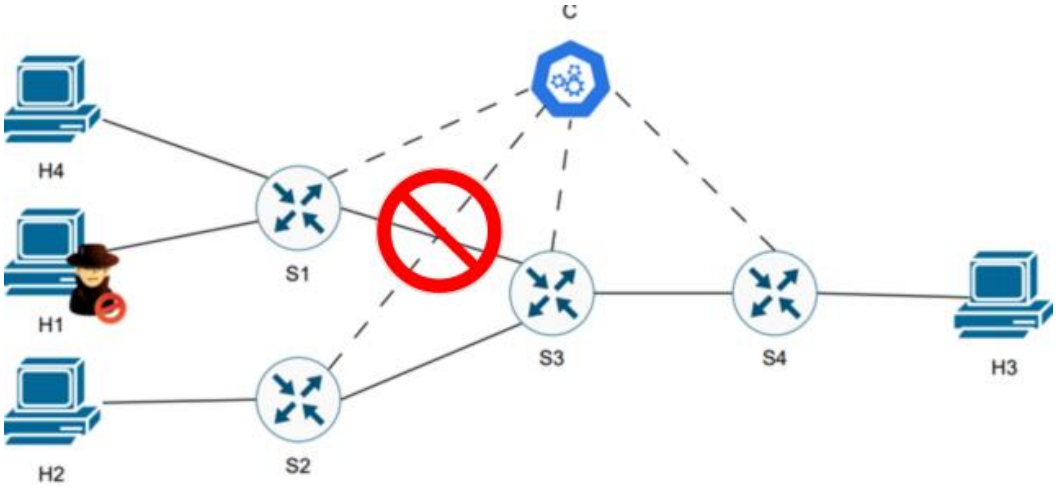

----------NON SUPERO LA SOGLIA-----------
Switch: 1, Port: 2, Rx Throughput: 0 bps, Tx Throughput: 0 bps, Alarm: False


----------NON SUPERO LA SOGLIA-----------
Switch: 1, Port: 3, Rx Throughput: 0 bps, Tx Throughput: 0 bps, Alarm: False
```

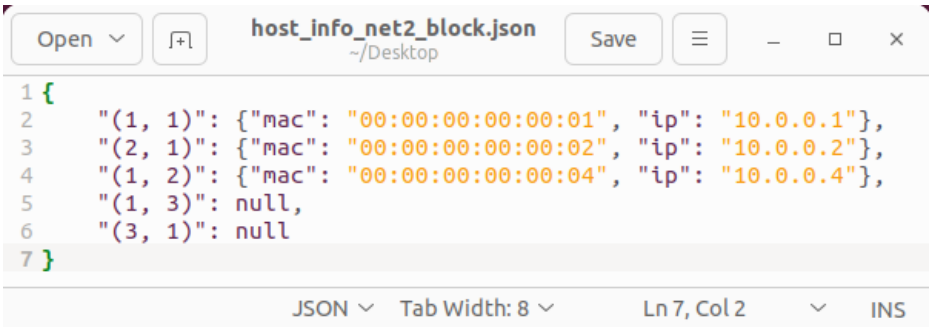
5.2.7 Soluzione 2 – Rimozione Link S1-S3

Fino ad ora abbiamo bloccato esclusivamente l'unica porta alla quale è collegato l'host attaccante; interessante è vedere il comportamento che si ottiene *rimuovendo l'intero link che collega lo switch S1 con lo switch S3*. Per poter agilmente bloccare il collegamento tra i due switch, è necessario aggiungere al file *json* le coppie (1,3) e (3,1) senza dover specificare indirizzo MAC ed indirizzo IP. In questo modo, quando il controller andrà a verificare la presenza della porta e dello switch, esso riuscirà a trovare lo switch 1 con porta 3 e lo switch 3 con porta 1.

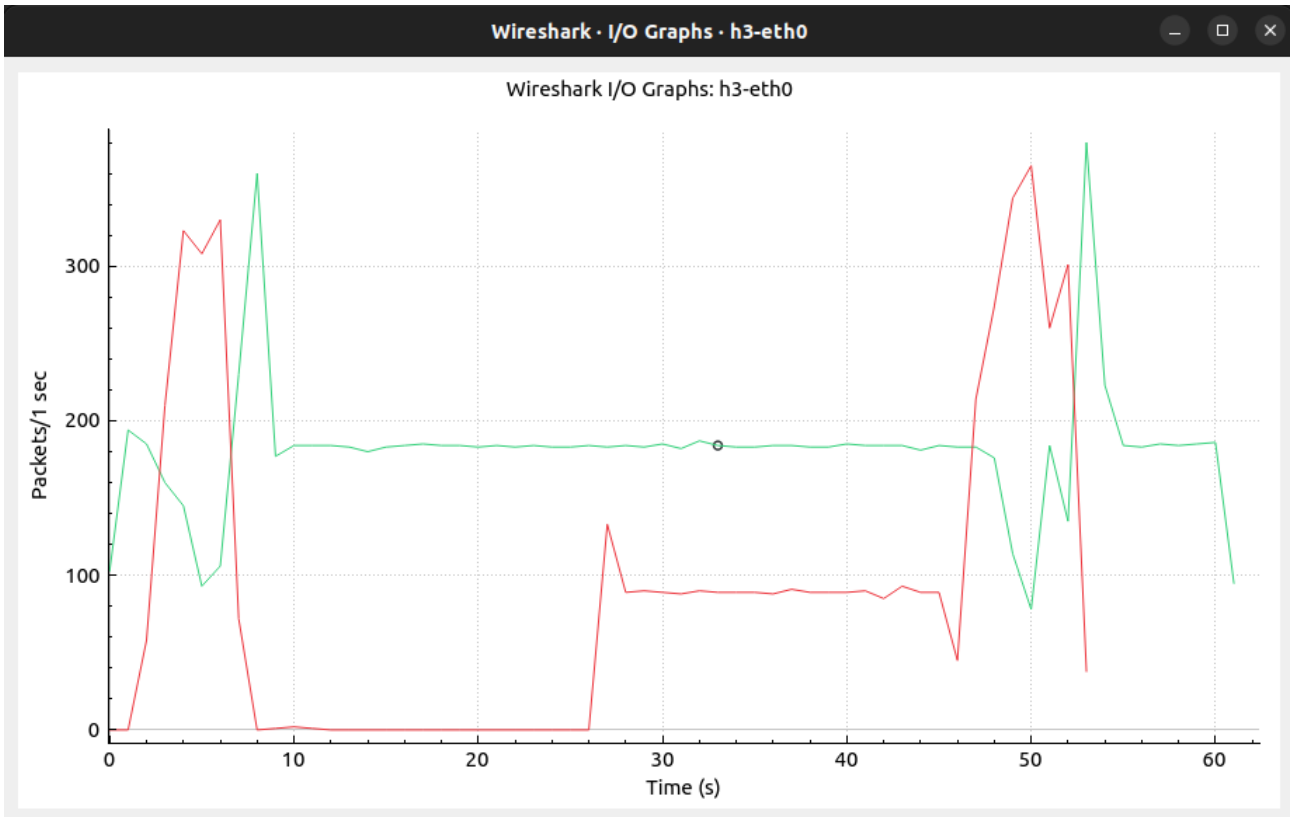
Questa risulta essere la nostra topologia di rete:



Il file *json* aggiornato è il seguente:



Eseguendo il controller con le modifiche apportate al file *json*, questa volta verrà anche bloccato il collegamento tra gli switch S1 ed S3. Il grafico che mostra l'andamento del traffico è simile alla precedente soluzione:



Nonostante i due grafici siano simili, il cambiamento più significativo si verifica quando si rimuove il collegamento tra i due switch. In questa situazione, l'host con l'indirizzo "10.0.0.4" rimane isolato a causa della presenza di un attaccante sullo stesso switch. Questo tipo di soluzione non è ottimale, poiché penalizzare alcuni utenti compromette la qualità del servizio (Quality of Service).

Quando si tenta di pingare l'host h3 dal terminale di h4 durante il blocco e lo sblocco del collegamento, si osserva una perdita dell'87.5% dei pacchetti. Questo accade perché l'host "10.0.0.4" non è in grado di comunicare durante l'interruzione del collegamento.

```
--- 10.0.0.3 ping statistics ---
8 packets transmitted, 1 received, 87.5% packet loss, time 7149ms
rtt min/avg/max/mdev = 229.841/229.841/229.841/0.000 ms
root@Ubuntu:/home/vboxuser/Desktop# ping 10.0.0.3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=112 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=181 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=119 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=113 ms
64 bytes from 10.0.0.3: icmp_seq=5 ttl=64 time=111 ms
^C
--- 10.0.0.3 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4019ms
rtt min/avg/max/mdev = 110.634/126.925/180.549/26.949 ms
root@Ubuntu:/home/vboxuser/Desktop#
```

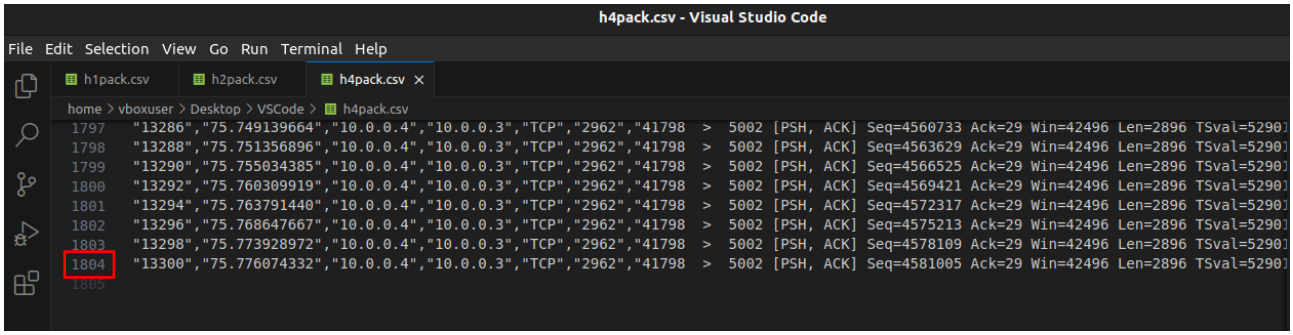
Inoltre, tramite wireshark, è possibile constatare che vengono rilevati dei salti dei pacchetti trasmessi tra 13 e 30 secondi:

```
2718 13.645790589  10.0.0.4   10.0.0.3   TCP   2962 41798 → 5002 [PSH, ACK
4575 30.494817140  10.0.0.4   10.0.0.3   TCP   1514 41798 → 5002 [ACK] Seq
```

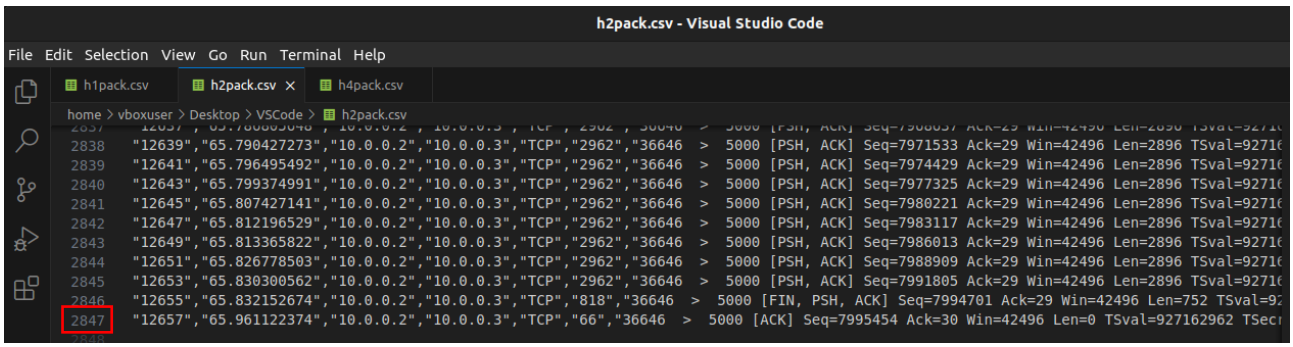
Stessa cosa accade tra 37 e 54 secondi, infatti le porte vengono bloccate esattamente due volte durante l'intera esecuzione:

```
7752 37.627971829  10.0.0.4   10.0.0.3   TCP   2962 41798 → 5002 [PSH, ACK
10311 54.421278364 10.0.0.4   10.0.0.3   TCP   1514 41798 → 5002 [ACK] Seq
```

Ulteriore verifica effettuata per constatare la perdita dei pacchetti dell'host h4 è stata quella di esportare tramite wireshark un file *csv* con tutte le informazioni dei pacchetti scambiati. Facendo ciò possiamo notare come la quantità di pacchetti tra h4 (host penalizzato) ed h2 (host avvantaggiato) risulti essere effettivamente minore per l'host "**10.0.0.4**".



Di contro, i pacchetti scambiati tra l'host h2 e l'host h3 risultano essere in numero maggiore, con un incremento percentuale di circa il **57,81%.**



Osservazioni:

La proposta di rimuovere il link tra lo switch S1 e lo switch S3 nella nostra topologia di rete è paragonabile alla soluzione implementata da **Piracy Shield**. Entrambe le strategie mirano a *mitigare* problemi di congestione o di sicurezza bloccando determinati percorsi di comunicazione. Tuttavia, questa soluzione presenta diverse criticità significative che meritano attenzione.

Infatti, Piracy Shield è una tecnologia progettata per prevenire l'accesso non autorizzato e la distribuzione di contenuti protetti da copyright. Funziona isolando nodi di rete sospetti o bloccando certi percorsi di comunicazione per impedire la diffusione di dati illegali. La rimozione del link tra S1 e S3 nella nostra rete

può essere paragonata all'approccio utilizzato dal Piracy Shield, la quale non è una soluzione ideale per diversi motivi e, difatti, la stessa strategia adottata da Piracy Shield è stata ampiamente criticata per gli stessi problemi:

- **Interruzione del Servizio per Utenti Legittimi**: La disconnessione del link tra S1 e S3 interrompe il normale flusso di traffico, colpendo anche gli utenti che non sono coinvolti in attività illecite. Questa interruzione non discriminatoria può causare disservizi per gli utenti legittimi, portando a una riduzione delle prestazioni della rete, tempi di latenza aumentati e potenziale isolamento da servizi e risorse della rete.
- **Limitazione dell'uso della topologia originale**: La rimozione del link porta all'impossibilità di utilizzare la piena capacità di trasmissione/banda offerta dalla topologia.
- **Spostamento della Congestione**: La rimozione del link non risolve la congestione, ma la sposta semplicemente altrove. Il traffico verrebbe reindirizzato attraverso percorsi alternativi, che potrebbero non avere la capacità necessaria per gestire il carico aggiuntivo, portando a una distribuzione inefficace delle risorse di rete e possibili congestioni in altri punti della rete.

Piracy Shield è stato ampiamente criticato per questi motivi. La sua implementazione ha spesso causato interruzioni del servizio per utenti legittimi, compromettendo la ridondanza della rete e generando congestioni inaspettate. Allo stesso modo, la rimozione del link tra S1 e S3 nella nostra rete, pur avendo intenti simili, presenta le stesse problematiche e non rappresenta una soluzione efficace.

5.2.8 Soluzione Alternativa – Blocco dell'indirizzo IP

Un'alternativa alla soluzione proposta precedentemente, nel caso in cui bloccare le porte degli switch risulti essere troppo invasivo in alcuni contesti applicativi, consiste nella creazione di una **blacklist di indirizzi IP**. In questo modo, non bloccheremo più le porte degli switch, ma gli indirizzi IP identificati come attaccanti. Questo approccio permette di mantenere più intatta la topologia di rete, poiché il controller interverrà solo sugli host considerati malevoli. La soluzione è molto vicina alla precedente ma meno invasiva, in quanto gli switch non vengono toccati.

La prima modifica effettuata è la creazione di un set chiamato *blacklist_ip*, il cui nome è già autoesplicativo: esso conterrà tutti gli indirizzi IP che non devono riuscire a comunicare, al fine di non intaccare gli altri utenti sulla rete.

N.B. è possibile utilizzare, invece degli indirizzi IP, gli indirizzi MAC al fine di mitigare un eventuale utilizzo di VPN da parte degli utenti malevoli. In questo modo, è possibile rendere più robusto l'approccio utilizzato con il file json, dato che la modifica di indirizzi MAC, seppur sempre possibile da effettuare, risulta essere più complicata.

```
12  class SimpleSwitch13(app_manager.RyuApp):
13      OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]
14
15      def __init__(self, *args, **kwargs):
16          super(SimpleSwitch13, self).__init__(*args, **kwargs)
17          self.mac_to_port = {}
18          self.datapaths = {}
19          self.stats_interval = 5  # Intervallo per raccogliere le statistiche dei porti (in secondi)
20          self.threshold = 2 * (10**6)  # Soglia in bit per secondo (2 Mbps)
21          self.alarm = {}
22          self.blacklist_ip = set()
23          self.entry_ports = {}  # Dizionario per tenere traccia delle porte di ingresso per ogni flusso
24          self.block_duration = 10  # Durata del blocco in secondi (prima prova: 5)
25          self.prev_stats = {}
26          self.host_info = {}  # Dizionario per tenere traccia degli host e delle loro porte e switch
27          self.label = set() # Dizionario per le etichette
28          self.load_host_info_json("host_info_net2.json")
```

Nella funzione *def port_stats_reply_handler* sono stati introdotti i seguenti cambiamenti:

- otteniamo l'indirizzo IP degli host, nel momento in cui la soglia viene superata
    - se l'indirizzo IP dell'attaccante non risulta essere ancora nella blacklist, allora chiamiamo la funzione *check_anfd_block_IP*, questa volta passando anche l'indirizzo IP come parametro.
    - se, invece, l'indirizzo IP risulta essere già bloccato, lo inseriamo nel set *label*, che serve per effettuare il controllo dinamico dello sblocco dell'indirizzo IP, dunque se trascorsi un certo numero di secondi (a nostra scelta) l'indirizzo non supera più la soglia, allora lo sblocchiamo.
- se la soglia non viene superata, l'indirizzo IP viene rimosso dalla blacklist.

```
102             attacker_info = self.host_info.get((datapath.id, port_no), {})
103             attacker_ip = attacker_info.get('ip', 'unknown')
104
105             if (dpid, port_no) in self.prev_stats:
106                 prev_rx_bytes, prev_tx_bytes = self.prev_stats[(dpid, port_no)]
107                 rx_bytes_diff = current_rx_bytes - prev_rx_bytes
108                 tx_bytes_diff = current_tx_bytes - prev_tx_bytes
109
110                 rx_throughput = (rx_bytes_diff * 8) / self.stats_interval
111                 tx_throughput = (tx_bytes_diff * 8) / self.stats_interval
112
113                 self.prev_stats[(dpid, port_no)] = (current_rx_bytes, current_tx_bytes)
114
115                 if rx_throughput > self.threshold:
116                     self.alarm[dpid, port_no] = True
117                     self.logger.info("----------SUPERO LA SOGLIA----------")
118
119                     if attacker_ip not in self.blacklist_ip:
120                         if (dpid, port_no) not in self.entry_ports:
121                             self.logger.info("****** CONTROLLO SE CHIAMARE LA FUNZIONE PER BLOCCARE L'INDIRIZZO IP ******")
122                             self.check_and_block_IP(datapath, port_no, attacker_ip)
123                             self.entry_ports[(dpid, port_no)] = True
124                         else:
125                             self.logger.info("****** TRAFFICO INTERMEDIO - POSSIBILE ATTACCO ******")
126                     else:
127                         self.logger.info("****** INDIRIZZO IP GIÀ BLOCCATO ******")
128                         if attacker_ip not in self.label:
129                             self.logger.info("AGGIUNTA ALLA LISTA DI LABEL")
130                             self.label.add(attacker_ip)
131                             print(self.label) # stampa di debug
132                 else:
133                     self.alarm[dpid, port_no] = False
134                     self.logger.info("----------NON SUPERO LA SOGLIA----------")
135
136                     if attacker_ip in self.blacklist_ip:
137                         self.blacklist_ip.remove(attacker_ip)
138                         if (dpid, port_no) in self.entry_ports:
139                             del self.entry_ports[(dpid, port_no)]
```

Altre modifiche risiedono nel controllo delle informazioni reperite dal file *json*, se il traffico è generato direttamente dall'host, allora chiamiamo la funzione *block_ip*, passando come parametro l'indirizzo IP dell'attaccante ed aggiungiamo il suo indirizzo IP alla blacklist.

Nella procedura *block_ip* avviene il cambiamento fondamentale del nuovo approccio:

- Viene creato un oggetto **OFPMatch** per specificare un match sui pacchetti con **eth_type=0x0800** (IPv4) e **ipv4_src=attacker_ip** (indirizzo IP sorgente dell'attaccante).
- Viene creato un elenco vuoto di azioni (**actions = []**), il che significa che i pacchetti che corrispondono a questo match saranno semplicemente scartati (bloccati).
- La funzione *add_flow* viene chiamata per aggiungere questa regola di flusso allo switch con priorità 2.

Infine, viene avviato il thread che gestisce l'operazione di sblocco dell'indirizzo IP.

N.B. Il *campo eth_type=0x0800* è utilizzato per identificare il tipo di protocollo di rete contenuto nel payload del frame Ethernet. Il valore 0x0800 è standardizzato dall'IEEE e specifica che il payload del frame Ethernet è un pacchetto IPv4.

```python
def check_and_block_IP(self, datapath, port_no, attacker_ip):
    dpid = datapath.id

    if (dpid, port_no) in self.host_info:
        self.logger.info("Traffico generato direttamente dall'host")
        self.logger.info("****** CHIAMO LA FUNZIONE PER BLOCCARE L'INDIRIZZO IP ******")
        self.block_ip(datapath, port_no, self.block_duration, attacker_ip)
        self.blacklist_ip.add(attacker_ip)
    else:
        self.logger.info("Traffico intermedio - possibile attacco")
        if (dpid, port_no) in self.entry_ports:
            del self.entry_ports[(dpid, port_no)]

def block_ip(self, datapath, port_no, duration, attacker_ip):
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser

    # accesso alle chiavi mac-ip
    attacker_info = self.host_info.get((datapath.id, port_no), {})
    attacker_mac = attacker_info.get('mac', 'unknown')

    # match per bloccare l'indirizzo IP dell'attaccante
    match = parser.OFPMatch(eth_type=0x0800, ipv4_src=attacker_ip)
    actions = []

    self.add_flow(datapath, 2, match, actions)

    # stampa delle informazioni sulla coppia switch-porta
    self.logger.info("Blocco del traffico dell'indirizzo ip %s con indirizzo MAC %s per %s secondi", attacker_ip, attacker_mac, duration)

    # Avvia un thread per sbloccare l'IP dopo una durata specifica
    threading.Thread(target=self.unblock_port_after_duration, args=(datapath, port_no, attacker_ip, duration)).start()
```

La modifica prevede il controllo sull'indirizzo IP, in particolare se esso non è presente nel set *label*, allora viene creato il nuovo match per sbloccare l'indirizzo IP e rimosso dalla blacklist, perché non più rilevato come malevolo. Inoltre, per preservare l'integrità, l'indirizzo IP viene anche rimosso dal set *label* oltre che dalla blacklist.

```python
def unblock_IP_after_duration(self, datapath, port_no, attacker_ip, duration):
    while True:
        time.sleep(duration)

        if attacker_ip not in self.label:
            ofproto = datapath.ofproto
            parser = datapath.ofproto_parser

            match = parser.OFPMatch(eth_type=0x0800, ipv4_src=attacker_ip)
            actions = [parser.OFPActionOutput(ofproto.OFPP_NORMAL)]

            self.add_flow(datapath, 2, match, actions)

            self.logger.info("Sblocco del traffico dall'IP %s dello switch %s", attacker_ip, datapath.id)

            self.blacklist_ip.discard(attacker_ip)
            if (datapath.id, port_no) in self.entry_ports:
                del self.entry_ports[(datapath.id, port_no)]
            break
        else:
            self.logger.info("Rimozione dalla lista dell'indirizzo IP %s", attacker_ip)
            self.label.remove(attacker_ip) # rimozione dell'indirizzo IP
            print(self.label) # stampa di debug
```

5.2.9 Soluzione Alternativa – Esecuzione con traffico variabile

Il grafico del traffico generato, come ci aspettavamo, è pressoché identico a quello della precedente soluzione, in quanto il comportamento è molto simile:



Anche il comportamento del controller rimane invariato; l'esperimento è stato eseguito con la generazione del bitrate variabile dell'host attaccante.

Inizialmente l'indirizzo IP viene bloccato perché supera la soglia di threshold:

```
----------SUPERO LA SOGLIA----------
****** CONTROLLO SE CHIAMARE LA FUNZIONE PER BLOCCARE L'INDIRIZZO IP ******
Traffico generato direttamente dall'host
****** CHIAMO LA FUNZIONE PER BLOCCARE L'INDIRIZZO IP ******
Blocco del traffico dell'indirizzo ip 10.0.0.1 con indirizzo MAC 00:00:00:00:00:01 per 10 secondi
IP Address: 10.0.0.1, Rx Throughput: 4318272 bps, Tx Throughput: 0 bps, Alarm: True


----------NON SUPERO LA SOGLIA-----------
IP Address: 10.0.0.4, Rx Throughput: 1072864 bps, Tx Throughput: 21542 bps, Alarm: False


----------NON SUPERO LA SOGLIA-----------
IP Address: unknown, Rx Throughput: 21542 bps, Tx Throughput: 5391136 bps, Alarm: False
```

Dopo il primo ciclo di controllo viene rilevato ancora come attaccante, e risulta già essere presente all'interno della blacklist.

```
-----------SUPERO LA SOGLIA----------
****** INDIRIZZO IP GIÀ BLOCCATO ******
AGGIUNTA ALLA LISTA DI LABEL
{'10.0.0.1'}
IP Address: 10.0.0.1, Rx Throughput: 4294147 bps, Tx Throughput: 67 bps, Alarm: True



----------NON SUPERO LA SOGLIA-----------
IP Address: 10.0.0.4, Rx Throughput: 1072864 bps, Tx Throughput: 29059 bps, Alarm: False
```

Dopo 10 secondi, l'indirizzo IP viene rimosso dal set label, perché il tempo di controllo è finito, in questo caso esso non viene sbloccato perché nell'arco temporale dei 10 secondi ha superato almeno una volta la soglia.

```
----------NON SUPERO LA SOGLIA-----------
IP Address: unknown, Rx Throughput: 57657 bps, Tx Throughput: 3077433 bps, Alarm: False



Rimozione dalla lista dell'indirizzo IP 10.0.0.1
set()
-----------SUPERO LA SOGLIA----------
****** INDIRIZZO IP GIÀ BLOCCATO ******
AGGIUNTA ALLA LISTA DI LABEL
{'10.0.0.1'}
IP Address: 10.0.0.1, Rx Throughput: 4340044 bps, Tx Throughput: 224 bps, Alarm: True



----------NON SUPERO LA SOGLIA-----------
IP Address: 10.0.0.4, Rx Throughput: 1072864 bps, Tx Throughput: 24406 bps, Alarm: False
```

Adesso l'attaccante sta generando traffico ad un rate non dannoso per gli altri utenti presenti nella rete, dunque, ci aspetteremo che il controller proceda con lo sblocco.

```
----------NON SUPERO LA SOGLIA-----------
IP Address: 10.0.0.1, Rx Throughput: 1076544 bps, Tx Throughput: 0 bps, Alarm: False



----------NON SUPERO LA SOGLIA-----------
IP Address: 10.0.0.4, Rx Throughput: 1072864 bps, Tx Throughput: 24288 bps, Alarm: False



----------NON SUPERO LA SOGLIA-----------
IP Address: unknown, Rx Throughput: 24288 bps, Tx Throughput: 1072864 bps, Alarm: False
```

Nei 10 secondi di controllo per la rimozione dell'indirizzo IP dalla blacklist, esso viene correttamente rimosso perché non risulta più essere un pericolo per gli altri host.

```
Sblocco del traffico dall'IP 10.0.0.1 dello switch 1
----------NON SUPERO LA SOGLIA-----------
IP Address: 10.0.0.1, Rx Throughput: 1081382 bps, Tx Throughput: 0 bps, Alarm: False
```

L'host 1 procede col generare nuovamente traffico dannoso per l'infrastruttura di rete; dunque, il controller procede nuovamente ad inserire il suo indirizzo nella blacklist.

```
----------SUPERO LA SOGLIA----------
****** CONTROLLO SE CHIAMARE LA FUNZIONE PER BLOCCARE L'INDIRIZZO IP ******
Traffico generato direttamente dall'host
****** CHIAMO LA FUNZIONE PER BLOCCARE L'INDIRIZZO IP ******
Blocco del traffico dell'indirizzo ip 10.0.0.1 con indirizzo MAC 00:00:00:00:00:01 per 10 secondi
IP Address: 10.0.0.1, Rx Throughput: 5566713 bps, Tx Throughput: 134 bps, Alarm: True
```

Nuovamente il controller rileva che l'indirizzo IP è già presente nella blacklist e lo aggiunge al set *label*.

```
----------NON SUPERO LA SOGLIA-----------
IP Address: unknown, Rx Throughput: 35174 bps, Tx Throughput: 5481923 bps, Alarm: False



----------SUPERO LA SOGLIA----------
****** INDIRIZZO IP GIÀ BLOCCATO ******
AGGIUNTA ALLA LISTA DI LABEL
{'10.0.0.1'}
IP Address: 10.0.0.1, Rx Throughput: 5660995 bps, Tx Throughput: 67 bps, Alarm: True



----------NON SUPERO LA SOGLIA-----------
IP Address: 10.0.0.4, Rx Throughput: 1072969 bps, Tx Throughput: 60508 bps, Alarm: False



----------NON SUPERO LA SOGLIA-----------
IP Address: unknown, Rx Throughput: 33964 bps, Tx Throughput: 1104553 bps, Alarm: False
```

Viene nuovamente rimosso dal set *label* dopo essere trascorsi i 10 secondi del controllo di rimozione dalla blacklist.

```
Rimozione dalla lista dell'indirizzo IP 10.0.0.1
set()
----------SUPERO LA SOGLIA----------
****** INDIRIZZO IP GIÀ BLOCCATO ******
AGGIUNTA ALLA LISTA DI LABEL
{'10.0.0.1'}
IP Address: 10.0.0.1, Rx Throughput: 5457782 bps, Tx Throughput: 67 bps, Alarm: True
```

Arriviamo alla fine della generazione del traffico e l'indirizzo dell'attaccante 10.0.0.1 non supera più la soglia.

```
----------NON SUPERO LA SOGLIA-----------
IP Address: 10.0.0.1, Rx Throughput: 0 bps, Tx Throughput: 796 bps, Alarm: False
```

Al termine della generazione del traffico viene nuovamente sbloccato.

```
Sblocco del traffico dall'IP 10.0.0.1 dello switch 1
----------NON SUPERO LA SOGLIA-----------
IP Address: 10.0.0.1, Rx Throughput: 112 bps, Tx Throughput: 0 bps, Alarm: False


----------NON SUPERO LA SOGLIA-----------
IP Address: 10.0.0.4, Rx Throughput: 0 bps, Tx Throughput: 112 bps, Alarm: False


----------NON SUPERO LA SOGLIA-----------
IP Address: unknown, Rx Throughput: 0 bps, Tx Throughput: 112 bps, Alarm: False


----------NON SUPERO LA SOGLIA-----------
IP Address: 10.0.0.2, Rx Throughput: 0 bps, Tx Throughput: 224 bps, Alarm: False
```

Conclusioni & Considerazioni

L'approccio che è stato adottato per la *detection* dell'attacco DoS è stato un approccio di tipo **Threashold-based**, ovvero, basato su un opportuno valore di soglia (come consigliato dalla traccia). Quando l'attività di rete supera questa soglia, viene generato un allarme per indicare un possibile attacco DoS. Questo approccio presenta vari *vantaggi* e *svantaggi*, descritti più dettagliatamente di seguito:

- **Vantaggi**:
  - *Semplicità implementativa* → le soglie possono essere facilmente configurate nei sistemi di monitoraggio senza la necessità di algoritmi complessi e senza risorse computazionali elevate
  - *Detection real-time* → rilevazione immediata quando l'attività di rete supera il livello impostato e risposta rapida a tali eventi anomali, minimizzando il tempo durante il quale l'attacco può causare danni alla rete
- **Svantaggi**:
  - *Presenza di falsi positivi* → dovuto ad una soglia troppo sensibile, generando allarmi anche per traffico legittimo che occasionalmente supera la soglia (esempio: nel caso della prima topologia, l'host h2 potrebbe essere identificato come un possibile attaccante)
  - *Difficoltà a determinare il corretto valore di threshold* → determinare il valore di soglia ottimale è molto complesso, poiché dev'essere abbastanza basso da rilevare attacchi ma abbastanza alto da evitare falsi positivi; inoltre, la possibile variazione del traffico di rete rende difficile identificare in maniera certa tale valore
  - *Impatto sui parametri della rete* → se la soglia è impostata in maniera tale da generare troppi allarmi, ciò può consumare risorse di calcolo e rete, portando ad un rallentamento della rete

In generale, data la semplicità della soluzione adottata, potrebbe essere utile combinare questa soluzione con altre metodologie di rilevazione, come algoritmi di apprendimento automatico, per migliorare l'accuratezza e ridurre i falsi positivi.

Approcci alternativi

Solitamente, tra gli approcci alternativi a quelli di tipo threashold-based, troviamo:

1) **Approcci features-based** → tali approcci utilizzano varie caratteristiche del traffico di rete per identificare attacchi DoS in modo più preciso rispetto ai metodi basati su soglia. Vediamone alcuni:
    - **ESIPs (Entropia degli IP sorgente)**: tale tecnica misura l'entropia degli indirizzi IP sorgente presenti nelle flow tables. Viene principalmente utilizzato per rilevare *attacchi M-DoS*, basandosi sull'idea che gli attaccanti utilizzano spesso la falsificazione degli IP sorgente per generare un gran numero di nuovi pacchetti. L'entropia degli IP sorgente aumenta significativamente durante un attacco M-DoS perché gli indirizzi IP sorgente diventano più *dispersi*.
    - **SFT (Similarità delle tabelle di flusso)**: valuta la correlazione tra le tabelle di flusso nel tempo. In condizioni normali, la distribuzione delle voci nelle tabelle di flusso è relativamente stabile. Tuttavia, durante un attacco M-DoS, molte nuove voci di flusso vengono aggiunte, causando variazioni significative nelle tabelle di flusso. Il SFT misura queste variazioni per rilevare anomalie.
2) **Approcci di Machine Learning (ML)** → tali approcci consentono, tramite l'apprendimento di dati storici del traffico di rete, di identificare schemi normali o anomali. Ciò porta ad una maggiore accuratezza, nonché a identificare pattern complessi che potrebbero non essere evidenti con metodi più semplici. Inoltre, i modelli di ML possono essere continuamente aggiornati con i nuovi dati, permettendo al sistema di adattarsi a nuove varianti di attacchi DoS che emergono nel tempo. Alcuni esempi di algoritmi di ML che è possibile adottare sono:
    - **KNN (K-Nearest Neighbor)**
    - **K-Means**
    - **Reti neurali**
    - **SVM**
    - **Modelli di Ensemble Learning**
    - **Alberi decisionali**

N.B. tutte le tecniche alternative illustrate risultano essere più complesse da implementare rispetto a quelle di tipo threshold-based.

Riferimenti

- Messaggi e strutture OpenFlow v1.3: documentazione di Ryu 4.34
- Monitoraggio del traffico: documentazione di Ryubook 1.0 (osrg.github.io)
- IEEE Xplore Full-Text PDF:
- IEEE Xplore Full-Text PDF:
- https://www.wired.it/article/piracy-shield-siti-ricorso-agcom-codice/